Pulse User Stories and Use Cases

Team Members

- Jacob Cohen
- Sujal Choukse
- Yatharth Bajaj

Stakeholder Map

Category	Stakeholders	Main needs
Primary	Farm operations manager; field technician	Receive field measurements at the hub. Preserve measurements during a radio outage.
Secondary	Deployment engineer; sensor integration developer; support technician	Reject invalid configurations. Add sensor types through stable contracts. Find delivery faults.
Hidden	Farm data steward; downstream analytics operator; radio compliance authority	Reject unapproved senders. Receive consistent records. Use approved radio settings.

The stories use the project records, architecture, current components, configuration examples, and automated tests as requirement sources.

User Stories

US-01 (primary)

As a farm operations manager, I want queued measurements to reach the farm hub after a radio outage, with enough time to transmit, store, and acknowledge them before their 3,600-second deadline, so that I can detect irrigation problems without visiting each sensor site.

US-02 (primary)

As a field technician, I want important samples to remain in the delivery queue until the hub stores them, so that a radio outage does not remove the farm's measurement history.

US-03 (secondary)

As a deployment engineer, I want the system to reject an invalid device configuration before installation, so that each field device starts with a supported hardware and software combination.

US-04 (secondary)

As a sensor integration developer, I want to add a sensor through the stable driver and decoder contracts, so that a new sensor does not require a change to the transport protocol.

US-05 (hidden)

As a farm data steward, I want the hub to accept records only from approved component identities, so that unapproved devices cannot add records to the farm data set.

INVEST Check

Story	INVEST check
US-01	**Independent:** The team can test delivery without a dashboard. **Negotiable:** The story does not select a radio or screen. **Valuable:** The manager receives measurements after an outage. **Estimable:** The Edge-to-Hub path defines the work. **Small:** The story covers one delivery result. **Testable:** A test can disconnect and restore the link.
US-02	**Independent:** The team can test queue behavior by itself. **Negotiable:** The story does not select a queue implementation. **Valuable:** The farm keeps important measurements. **Estimable:** The acknowledgement and queue limits define the work. **Small:** The story covers one stored sample. **Testable:** A test can inspect the queue, database, and status.
US-03	**Independent:** The Builder can validate a file without field hardware. **Negotiable:** The story does not select an input tool. **Valuable:** The engineer finds faults before installation. **Estimable:** The configuration rules define the work. **Small:** The story covers one validation result. **Testable:** A test can check the process result and error text.
US-04	**Independent:** The team can add one driver and one decoder. **Negotiable:** The story does not select a sensor model. **Valuable:** The project can support another sensor. **Estimable:** The driver and decoder contracts define the work. **Small:** The story covers one sensor type. **Testable:** Protocol tests can compare the record before and after transport.
US-05	**Independent:** The team can test identity checks with test keys. **Negotiable:** The story does not select a key tool. **Valuable:** The data set excludes unapproved senders. **Estimable:** The trust table defines the work. **Small:** The story covers one sender decision. **Testable:** A test can use approved and unapproved identities.

Use Cases

UC-01: Deliver Queued Telemetry

- **Expands:** US-02
- **Primary actor:** Field technician
- **Secondary actors:** Edge component, field Node, hub Node, and Hub component

Preconditions

1. The field configuration uses `ack = "stored"`.
2. The outbound queue has space for at least one reliable message.
3. The field and hub Nodes have approved peer identities.
4. One sensor has a sample interval of 60 seconds.
5. The Hub database has no row for the test message identifier.
6. The radio link is available before the test starts.

Main Success Flow

1. The field technician starts the field Node, Edge, hub Node, and Hub.

2. The system validates each configuration and registers the Hub destination.

3. The field technician disconnects the field link before the next sample time.

4. The Edge collects one sample. The field Node keeps the message in its reliable queue.

5. The field technician restores the link with enough time for transmission, storage, and acknowledgement before the message expires.

6. The field Node sends the queued message with its original message identifier.

7. The hub Node checks the sender identity and gives the message to the Hub.

8. The Hub validates the message and commits one database row.

9. The Hub sends a `STORED` acknowledgement after the commit.

10. The field Node removes the acknowledged message from its reliable queue.

Alternate Flow: Duplicate Transmission

1. The field Node retransmits the message with the same message identifier before it receives the acknowledgement.

2. The Hub finds the existing duplicate key and does not create a second logical record.

3. The Hub sends a `STORED` acknowledgement for the committed record.

4. The field Node removes the acknowledged message from its reliable queue.

Exception Flow: Unapproved Sender

1. The hub Node receives the queued message from an identity that is not in its trust table.

2. The hub Node rejects the message as `UNAUTHORIZED`.

3. The Hub does not receive the message and does not add a database row.

4. The field Node keeps the message until a valid acknowledgement or the configured expiry.

Postcondition

The Hub stores exactly one row for an approved message. The field queue has no copy after the `STORED` acknowledgement.

UC-02: Validate a Device Configuration

- **Expands:** US-03
- **Primary actor:** Deployment engineer
- **Secondary actors:** Builder and sensor driver registry

Preconditions

1. The deployment engineer can run the Builder.
2. The `sim-env` driver is in the selected Edge build.
3. The engineer has a device configuration file that follows schema version 1.
4. The source configuration file is under version control.

Main Success Flow

1. The deployment engineer selects the target platform and the device configuration file.
2. The engineer runs the Builder validation command.
3. The Builder parses the file and checks all required sections.
4. The Builder checks the platform, local carrier, acknowledgement mode, driver name, and sample interval.
5. The Builder reports a successful result and returns process status 0.
6. The deployment engineer marks the unchanged file as ready for the next build step.

Alternate Flow: List Included Drivers

1. The deployment engineer does not know which sensor drivers are in the selected build.
2. The engineer runs the Builder command that lists drivers.
3. The Builder prints each included driver name.
4. The engineer selects `sim-env` and returns to step 2 of the main flow.

Exception Flow: Driver Is Not Included

1. The configuration names a driver that is not in the selected build.
2. The Builder rejects the configuration and returns a nonzero process status.
3. The Builder reports the invalid driver name and the available driver names.
4. The Builder leaves the source configuration file unchanged.

Postcondition

A successful result identifies one unchanged configuration as valid for the selected platform. A failed result does not approve the configuration.

Acceptance Criteria

UC-01 Acceptance Criteria

AC-01.1 - Main Success Flow

Given an approved field identity, an empty Hub database, and a field configuration that requests `STORED`, When the technician disconnects the link for one sample and restores it with enough time for transmission, storage, and acknowledgement before the 3,600-second expiry, Then the Hub contains exactly one row for the message identifier and the field queue contains zero copies after acknowledgement.

AC-01.2 - Alternate Flow

Given the Hub already stored a message with one origin and message identifier, When the Hub receives the same message two more times, Then the Hub contains exactly one logical row for that origin and message identifier and returns `STORED`.

AC-01.3 - Exception Flow

Given the hub Node trust table does not contain the field identity, When that field identity sends one message to the Hub destination, Then the hub Node reports `UNAUTHORIZED` and the Hub adds zero rows for the message identifier.

UC-02 Acceptance Criteria

AC-02.1 - Main Success Flow

Given a schema version 1 Linux configuration with a 128-character hexadecimal peer identity, `loopback_tcp`, `stored`, and the included `sim-env` driver, When the deployment engineer runs the Builder validation command, Then the command returns process status 0 and reports zero validation errors.

AC-02.2 - Alternate Flow

Given the selected Edge build includes the `sim-env` driver, When the deployment engineer runs the Builder driver-list command, Then the command output contains `sim-env` exactly once.

AC-02.3 - Exception Flow

Given a configuration that names `missing-driver` and a source file with a recorded hash, When the deployment engineer runs the Builder validation command, Then the command returns a nonzero status, names `missing-driver`, lists available drivers, and preserves the source file hash.

Pulse - D0 High-Level Design

Assignment 4: Design Diagrams, D0

Team: Jacob Cohen, Sujal Choukse, Yatharth Bajaj
Advisor: Jeffrey Gundler

1. Title, goal statement, and conventions

Project title: Pulse - platform-independent IoT communication for resilient farm sensor networks.

Goal statement: Move every farm sensor reading from a field device to a trusted Hub over any available link, and keep each reading until the Hub stores it.

Basic input: a measurement from a field sensor (soil, water, or environment). Each device also gets one configuration file before it starts.

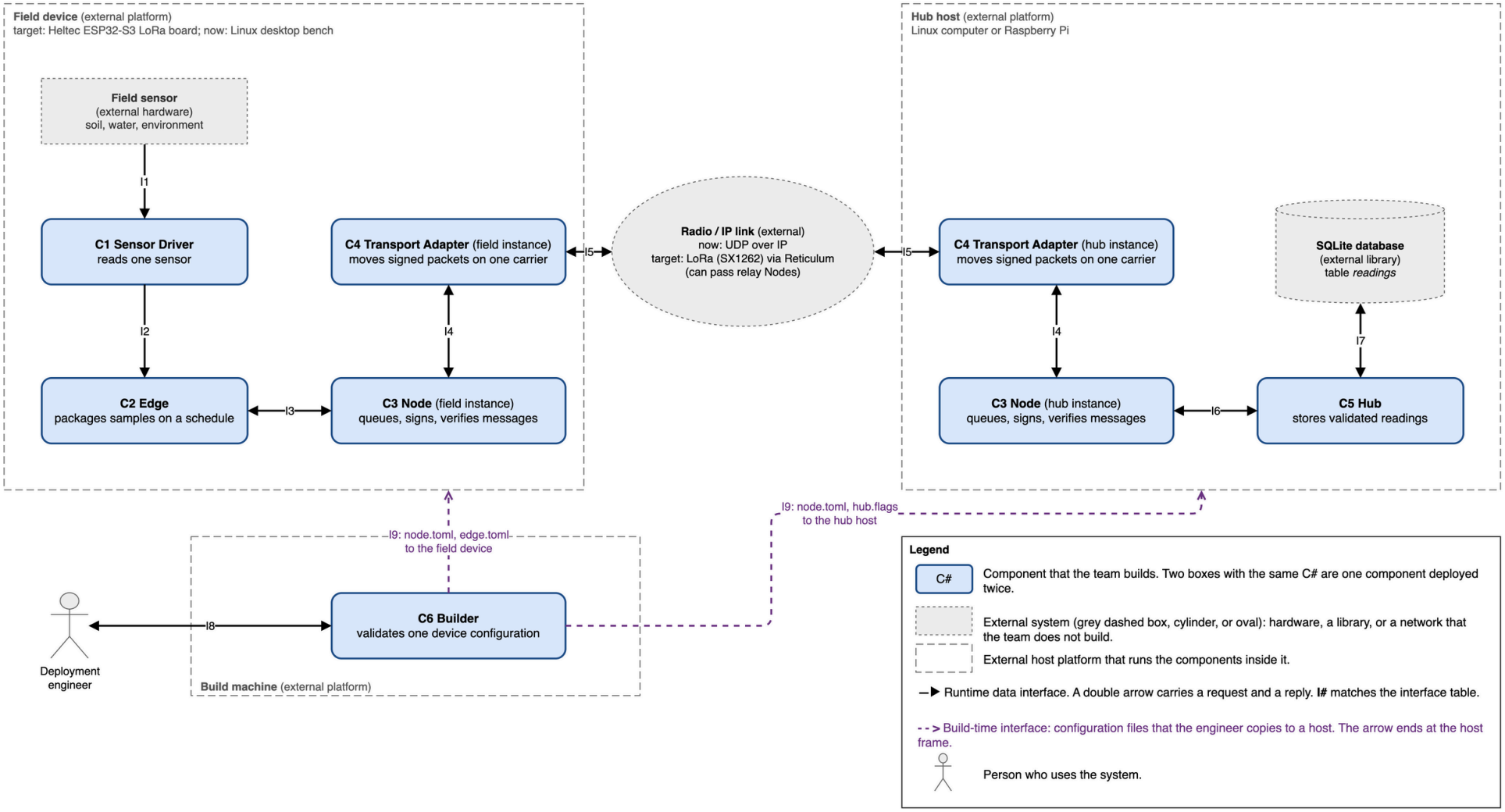
Basic output: a stored row for each reading in the Hub database, with the sender identity and the collection time. The field device gets a `STORED` receipt back.

Conventions. Both diagrams use the same symbols, and each diagram has its own legend.

Symbol	Meaning
Blue rounded box, **C#**	A component that we build. Two boxes with the same C# are one component that runs on two hosts.
Grey dashed box, cylinder, or oval	An external system that we do not build: hardware, a library, or a network.
Large dashed frame	A host platform. The components inside the frame run on it.
Stick figure	A person who uses the system.
Solid arrow, **I#**	A runtime interface. A double arrow carries a request and a reply. I# matches the table in part 4.
Purple dashed arrow	Configuration files that the engineer copies to a host.
Red dashed arrow	A receipt or an error that goes back to the sender.
A# / **B#** box	A stage in the data-flow diagram. The box names the component that does the stage.

2. Block diagram (D0)

Pulse - platform-independent IoT communication for resilient farm sensor networks
D0 High-Level Design - Block Diagram (Team: Jacob Cohen, Sujal Choukse, Yatharth Bajaj)
Goal: move every farm sensor reading from a field device to a trusted Hub over any available link, and keep each reading until the Hub stores it.



Pulse has six components (C1 to C6) and nine interfaces (I1 to I9). The Node (C3) and the Transport Adapter (C4) run on every device, so they appear once on the field device and once on the hub host.

Today the system runs on Linux computers and uses UDP over IP between devices. The target field device is a Heltec ESP32-S3 board that sends over LoRa through Reticulum.

3. Component responsibility table

ID	Component	Responsibility	In	Out	Owner
C1	Sensor Driver	Reads one sensor type and returns its raw reading.	I1	I2	Sujal Choukse
C2	Edge	Turns each scheduled sample into a delivery request for the local Node.	I2, I3, I9	I3	Yatharth Bajaj
C3	Node	Keeps each outgoing message until the requested receipt arrives or the message expires.	I3, I4, I6, I9	I3, I4, I6	Jacob Cohen
C4	Transport Adapter	Moves signed packets between the Node and one carrier: UDP now, LoRa later.	I4, I5	I4, I5	Jacob Cohen
C5	Hub	Stores each valid reading once, then confirms that it is stored.	I6, I7, I9	I6, I7	Yatharth Bajaj
C6	Builder	Checks a device configuration file and splits it into one file for each component.	I8	I8, I9	Sujal Choukse

The Node also checks sender identities, because a message is only delivered when it comes from a trusted sender. The shared protocol package (message definitions and test vectors) is not a component, because it does not run. It defines the message formats for I3, I5, and I6.

Traceability. Each user story maps to at least one component, and each component maps to at least one story.

User story	Components
US-01: readings reach the Hub after an outage	C2, C3, C4, C5
US-02: samples stay queued until the Hub stores them	C3, C5
US-03: reject an invalid configuration before installation	C6
US-04: add a sensor through the driver and decoder contracts	C1, C5
US-05: accept records only from approved identities	C3

4. Interface specification table

ID	From → To	Inputs	Outputs	Format	Protocol	On error (handled by)
I1	Field sensor → C1	physical quantity: analog or digital signal	reading: bytes	Value native to the sensor	Sensor bus (I2C, UART, or ADC). The `sim-env` driver simulates it today.	The read fails and C1 returns `DriverError`. (C1)
I2	C1 → C2	`sample()` call	`payload`: bytes; `encoding`: string, for example `application/json`	Driver result	Function call through the `SensorDriver` contract	C2 logs the error and skips that sample. The schedule continues. (C2)

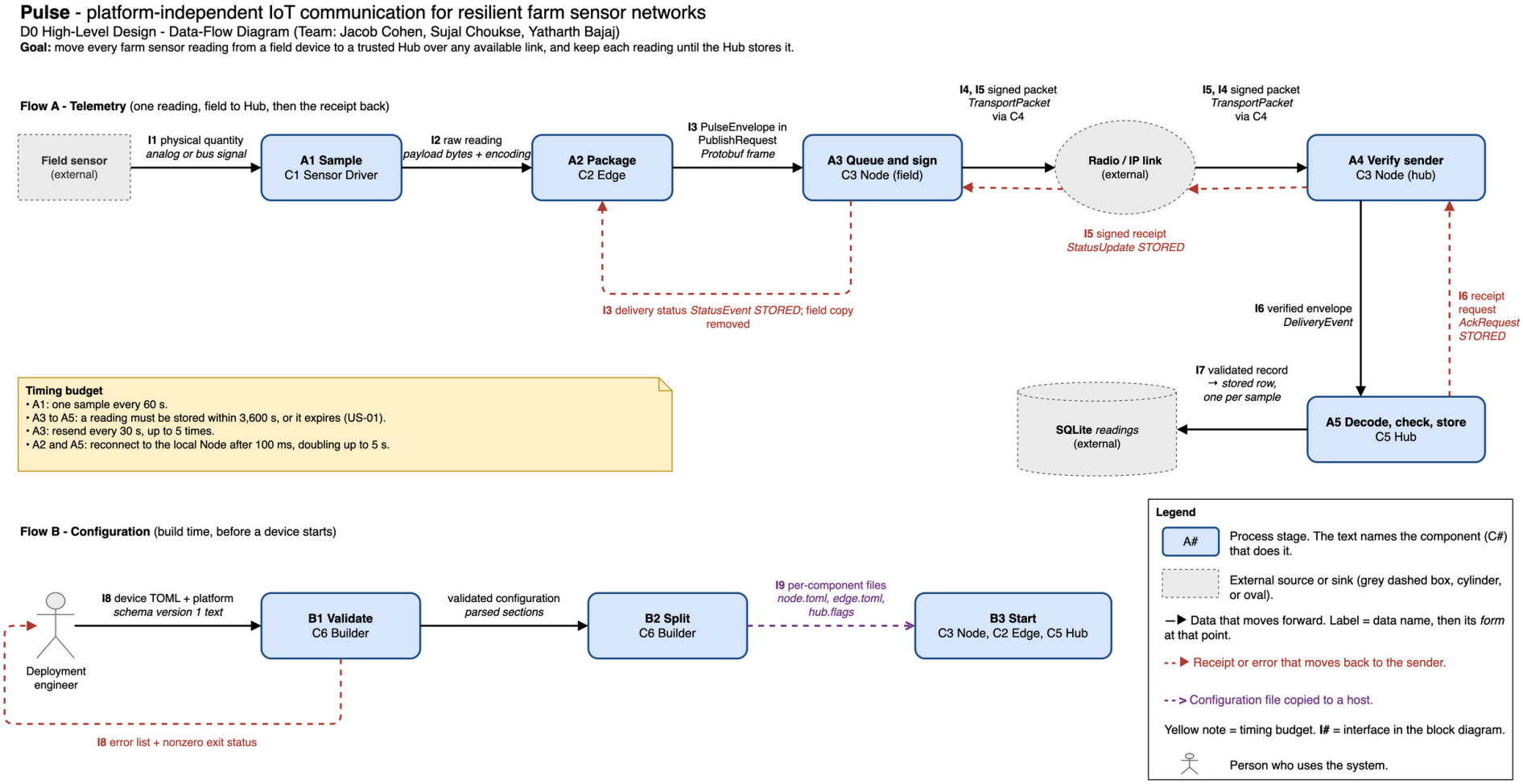
ID	From → To	Inputs	Outputs	Format	Protocol	On error (handled by)
I3	C2 ↔ C3	`PublishRequest{destination: string, message: PulseEnvelope, ack: AckMode, want_acceptance: bool}`	`Response{ok: bool, error: ErrorDetail}`; `StatusEvent{message_id: string, status: DeliveryStatus, detail: string}`	Protobuf `RpcEnvelope` with a 4-byte length prefix, 1 MiB maximum	TCP on the loopback address only	The Node returns `QUEUE_FULL` or `INVALID_MESSAGE`, or sends a bad frame. The connection closes and the Edge reconnects after 100 ms, doubling up to 5 s. (C3 detects, C2 recovers)
I4	C3 ↔ C4	outgoing `TransportPacket` bytes and the next peer	incoming `TransportPacket` bytes and the source	Serialized `TransportPacket`	Function call through the `Transport` contract	The send fails. The Node keeps the message and tries again at the next retry time. (C3)
I5	C4 ↔ C4 over the radio or IP link	`TransportPacket{payload: SignedPayload, sender_identity: 32 bytes, signature: 64 bytes, ttl: uint32}`	the same packet at the receiver	Protobuf, signed with ed25519	One UDP datagram per packet today; a Reticulum packet over LoRa later	The receiver drops packets from unknown peers or with bad signatures. The sender resends every 30 s, up to 5 times, and marks the message `EXPIRED` after 3,600 s. (receiving C3 drops, sending C3 recovers)
I6	C3 ↔ C5	`SubscribeRequest{service: string}`; `AckRequest{message_id: string, status: STORED}`	`DeliveryEvent{destination: string, message: PulseEnvelope, authenticated_sender: bytes}`	Protobuf `RpcEnvelope`, same framing as I3	TCP on the loopback address only	The Hub reconnects and subscribes again after 100 ms, doubling up to 5 s. An event that is lost gets no receipt, so the sender resends it. (C5, sending C3)
I7	C5 ↔ SQLite	one row per sample: `origin_id`, `message_id`, `sensor_id`, `encoding`: text; `payload`, `authenticated_sender`: blob; `collected_at_ms`, `received_at_ms`: integer	`STORED` or `DUPLICATE`	Row in table `readings`, keyed on (`origin_id`, `message_id`, `sensor_id`)	JDBC; `INSERT OR IGNORE` in one transaction	The transaction rolls back and the Hub sends no receipt, so the sender resends. (C5)
I8	Engineer ↔ C6	configuration file: TOML, `schema_version = 1`; platform: `linux`, `windows`, or `esp32`	validation report: text; exit status: integer	TOML file and command-line arguments	Command-line program on the build machine	The Builder lists each error, exits with a nonzero status, and leaves the file unchanged. (C6)
I9	C6 → C2, C3, C5	validated configuration	`node.toml`, `edge.toml`: TOML; `hub.flags`: text	TOML and command-line flags	File copy to the host	The component stops at startup with an error, and systemd restarts it. (receiving component)

The main enumerations are `AckMode` (`ACK_NONE`, `ACK_RECEIVED`, `ACK_STORED`) and `DeliveryStatus` (`RECEIVED`, `STORED`, `EXPIRED`, `FAILED`).

Example payload. This `PulseEnvelope` carries one soil moisture reading over I3 and I5. It is shown as JSON here and sent as Protobuf.

```
{
  "envelope_version": 1,
  "message_id": "11111111-2222-3333-4444-555555555555",
  "origin_id": "field-01",
  "collected_at_ms": 1788912000000,
  "samples": [
    { "sensor_id": "soil-1", "encoding": "application/json",
      "payload": "{\"moisture\":0.42}" }
  ]
}
```

5. Data-flow diagram



Flow A: telemetry. A reading changes form at each stage.

1. **A1 Sample (C1):** the sensor signal becomes a raw reading.
2. **A2 Package (C2):** the raw reading becomes a `PulseEnvelope` with a message ID, the origin ID, and the collection time.
3. **A3 Queue and sign (C3, field):** the envelope waits in the queue and is sent as a signed packet.
4. **A4 Verify sender (C3, hub):** the packet becomes a verified envelope with the sender identity.
5. **A5 Decode, check, store (C5):** the envelope becomes a validated record, then one stored row per sample.

After the row is stored, the Hub asks its Node to send a `STORED` receipt. The receipt goes back over the link. The field Node then deletes its copy and tells the Edge.

Flow B: configuration. The engineer gives a configuration file to the Builder. The Builder checks it (B1) and splits it (B2) into `node.toml`, `edge.toml`, and `hub.flags`. Each component starts with its own file (B3). If the file is not valid, the engineer gets a list of errors.

Timing budget.

Item	Value
Sample interval	60 s
Time before an undelivered message expires (US-01)	3,600 s
Resend interval	30 s, up to 5 times
Reconnect to the local Node	100 ms, doubling up to 5 s

6. Architecture pattern and justification

Pulse is a **pipeline**, with **layers** inside each device and an **embedded sensor loop** on the field device.

Pattern	Where it applies
Pipeline	The telemetry path: sample, package, queue and sign, carry, verify, store. Each stage has one job and passes one data form to the next.
Layered	Inside each device: the Edge or the Hub on top, then the Node, then the Transport Adapter, then the carrier. Each layer uses only the layer below it.
Embedded sensor-actuator	The field device reads its sensors on a fixed schedule. The first release has sensors only.
Client-server (local)	On each host, the Node is a local server, and the Edge and the Hub are its clients.

Why these patterns fit, using the five criteria.

1. **Fit to the problem.** Readings move one way through fixed stages, and receipts move back. A pipeline matches this. The layers let us move from UDP to LoRa by changing only C4.
2. **Team skills.** We know Rust, Java, Linux, and networking. Fixed contracts between the stages let each person build and test one part alone with shared test vectors.
3. **Performance and timing.** A reading every 60 s with a 3,600 s deadline is a low data rate. The real limit is LoRa airtime, so messages use compact Protobuf and the design adds no extra network services.
4. **Scalability.** One Hub collects from many field devices. A new device only needs a configuration file and a trusted identity, and relay Nodes extend the range. One farm fits easily in one SQLite database.
5. **Hardware and project constraints.** The field device is a battery-powered ESP32-S3 board, so its software stack must stay small, and battery devices do not relay packets. We have a $200 budget and one LoRa board, so the design needs no cloud service or paid software. Every packet is signed, and the local server accepts only connections from the same host.

Rejected patterns.

- **Microservices.** Each service would need its own network API and deployment. An ESP32-S3 cannot run this, a team of three cannot maintain it, and a farm has no reliable network to connect the services.
- **Cloud client-server.** Field devices would send straight to a cloud server. This fails on a farm with no internet access, which breaks the goal of delivery over any available link, and it adds a monthly cost.

7. Decision log

ID	Decision	Alternatives	Why this option
DL-1	Run the Node, the Edge, and the Hub as separate programs.	One program per device; one program for the whole system.	If the Edge or the Hub fails, the Node keeps forwarding. Each program restarts on its own, and the Hub can be rewritten in another language.
DL-2	Use Protobuf over loopback TCP for the local connection.	gRPC; HTTP with JSON.	Protobuf is small enough for LoRa and generates code for Rust and Java. gRPC and HTTP need a server stack that an ESP32 cannot run. Loopback only means that no other machine can connect.
DL-3	Put the carrier behind a Transport Adapter, and prove the system on UDP before adding LoRa.	Build on LoRa hardware from the start.	We can test every other component on laptops now. Our advisor agreed to prove the data path on computers before full hardware support.
DL-4	Send the `STORED` receipt only after the database commit, and remove duplicates with the key (`origin_id`, `message_id`, `sensor_id`).	Send the receipt as soon as the packet arrives.	An early receipt loses the reading if the Hub stops before it commits. Resends keep the same message ID, so a resend never creates a second row.
DL-5	Sign every packet with an ed25519 key, and accept only peers in a trust table.	No authentication; TLS.	US-05 allows only approved senders. TLS needs a connection, and LoRa packets have none. The signing follows Reticulum's model, so we do not invent our own cryptography.
DL-6	Store readings in SQLite on the hub host.	PostgreSQL; a cloud database.	SQLite needs no server or network and runs on a Raspberry Pi. One farm does not need more. A cloud database fails without internet access.

Pulse - Detailed System Design (D1)

Assignment 6 - Detailed System Design, Part 1 (D1)

Part of the Assignment 4 design-diagram collection. D2 (Assignment 7) updates the capacity, retry scheduling, carrier limits, and failure handling below.

1. Header, scope, and conventions

Project title: Pulse - platform-independent IoT communication for resilient farm sensor networks.

Goal statement: Move every farm sensor reading from a field device to a trusted Hub over any available link, and keep each reading until the Hub stores it.

Team: Jacob Cohen, Sujal Choukse, Yatharth Bajaj. **Advisor:** Jeffrey Gundler. **Date:** October 6, 2026.

This document details the Node (C3) and Hub (C5); D2 details the Sensor Driver (C1), Edge (C2), Transport Adapter (C4), and Builder (C6).

The Node owns reliable delivery and sender checks. The Hub owns the data model and the storage transaction. Jacob owns C3. Yatharth owns C5. Sujal checks sensor formats with both owners. The shared Protobuf package defines I3, I5, and I6. It does not run as a component.

Figure conventions

Figure 1 uses rectangles for Hub tables. Each rectangle lists the table name and its key fields. Solid lines show foreign-key relationships. Labels give the count at both ends: `1` means exactly one, `0..*` means zero or more, and `1..*` means one or more. `PK` marks a primary key. `FK` marks a foreign key. One composite key uses all marked fields together. Lines do not show message flow. The figure uses no person or hardware symbols. The SQL source gives the full constraints. The Hub transaction enforces the minimum of one Reading per Envelope.

Basis and changes from D0

The project title, goal, component IDs, and reading key stay the same as D0 [P1]. The following changes make D0 precise.

Item	D1 decision and basis
Retry limit	Retry every 30 seconds until the 3,600-second deadline. Five retries alone do not cover US-01. This changes D0's retry limit.
Queue key	Each field Node uses `(message_id, destination)`. This prevents one destination's receipt from removing another destination's copy.
Hub model	Origin, Sensor, and Envelope tables support the existing Reading key. The Hub checks duplicate content before sending a receipt.
Carrier and identity	Current code uses Reticulum over TCP, a 16-byte identity hash, and no Pulse TTL. D0 names UDP and a 32-byte sender key.
Build status	These refinements define target behavior. Current code still uses message-only queue keys, five retries, and a flat readings table [P3].

The queue stays in memory in D1. Outage tests keep both Nodes running. Recovery after a Node restart depends on the durable queue design in D2. A local acceptance response confirms memory admission. It does not confirm storage at the Hub.

2. Data model, the D1 diagram

Figure 1 details C5. All four tables use SQLite on the Hub host.

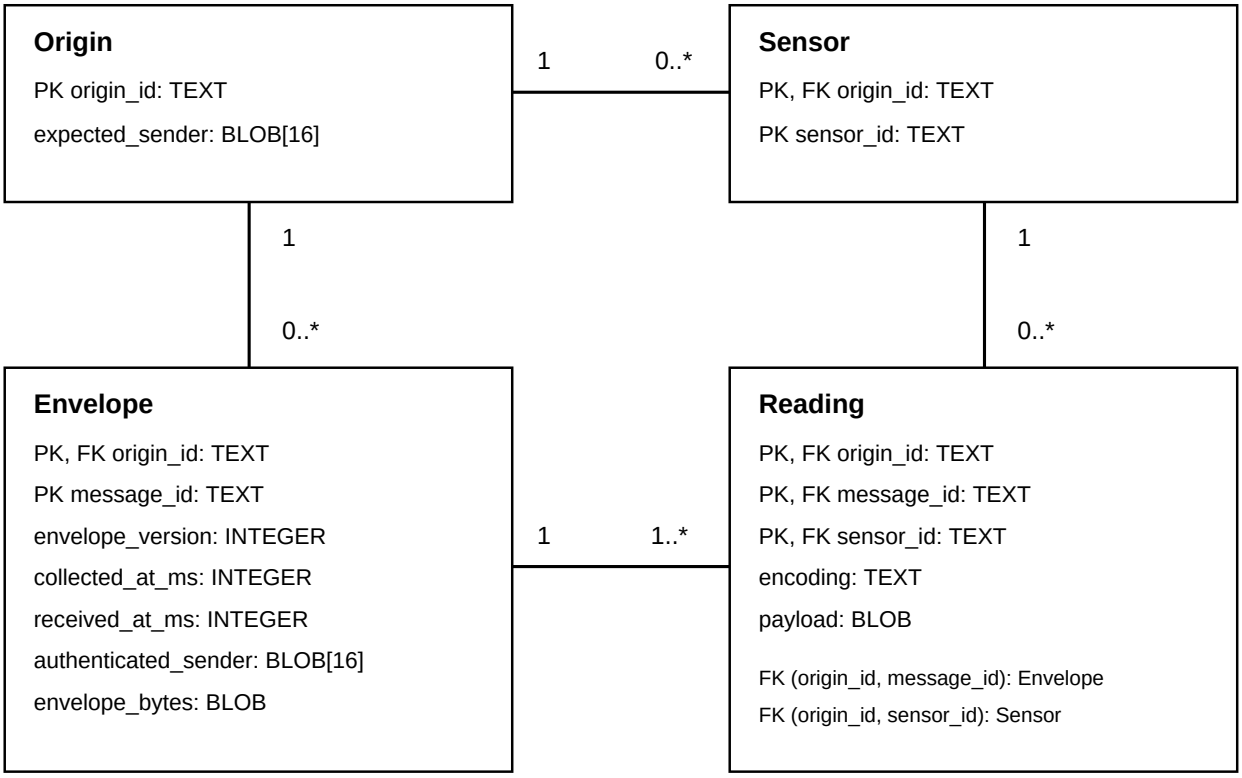
Pulse | Figure 1. Hub data model

Goal: move every farm reading to a trusted Hub and keep it until stored.

Legend: boxes = tables; lines = foreign keys; 1 / 0..* / 1..* = cardinalities.

Figure 1. C5 Hub: relational data model

Target D1 schema. PK = primary key; FK = foreign key.



A2 enforces one or more Readings per Envelope in one transaction. Lines show relationships, not message flow.

Structural decisions

Decision	Reason and requirement
Envelope is an entity	One publication groups samples into one commit and one receipt. Its stored content identifies conflicting retries. US-01, US-02; I6, I7.
Sensor is an entity	An origin can use one sensor across many messages. An encoding belongs to each Reading because formats can change. US-04.
One-to-many links	Each Envelope has one Origin. Each Reading has one Envelope and one Sensor. Reading joins their histories. US-02, US-04.
Relational store	Keys, foreign keys, and transactions protect a complete publication. A plain file needs custom checks and recovery. US-02, US-05.

An Origin represents a configured device name. Its expected sender hash binds that name to one identity. The Node trust table remains the authority. The Hub imports the same binding before it accepts remote data. One identity can own several Origin

names. Key changes require a controlled configuration update; D1 does not permit two active keys for one Origin.

Key attributes, constraints, and indexes

Table	Key attributes and rules
Origin	`origin_id: TEXT`, PK; `expected_sender: BLOB`, exactly 16 bytes. The name contains 1-64 ASCII characters. The Origin row exists before delivery.
Sensor	`(origin_id, sensor_id): TEXT`, PK; `origin_id` is an FK. Sensor rows exist before delivery. Both names contain 1-64 ASCII characters.
Envelope	`(origin_id, message_id): TEXT`, PK; `origin_id` is an FK. The stored fields are `envelope_version: INTEGER = 1`, `collected_at_ms: INTEGER >= 0`, `received_at_ms: INTEGER > 0`, `authenticated_sender: BLOB` of 16 bytes, and `envelope_bytes: BLOB`.
Reading	`(origin_id, message_id, sensor_id): TEXT`, PK. The first two fields form an FK to Envelope. `(origin_id, sensor_id)` forms an FK to Sensor. The Hub stores `encoding: TEXT` and `payload: BLOB` unchanged.

All fields are required. `collected_at_ms = 0` means the sensor time is unknown. All other times use UTC milliseconds since the Unix epoch. The receiver clock supplies `received_at_ms`. Arrival time does not replace collection time. Payload units belong to the decoder contract. For example, the moisture value is a fraction from 0 to 1.

Index	Lookup and reason
Envelope PK	Find a duplicate by origin and message in logarithmic time. US-02; I7.
Reading PK	Enforce D0's unique sample key and fetch a whole envelope. US-02, AC-01.2.
Sensor PK and Origin PK	The keys support sensor and identity checks. US-04, US-05.
Envelope `(origin_id, collected_at_ms, message_id)`	Read a device's time range without a full scan. The final field resolves equal times. US-01.
Reading `(origin_id, sensor_id, message_id)`	Fetch sensor history and check references before sensor removal. US-04.

Our planning period is 30 days. The Hub retains each Envelope and its Readings for at least 3,600 seconds after receipt. It retains duplicate keys throughout the retry period. D2 defines retention and disk monitoring. SQLite uses `foreign_keys=ON`, `journal_mode=WAL`, `synchronous=FULL`, and a 5,000 ms busy timeout. FULL adds a disk sync at each commit in WAL mode [W1].

C3 state model

`Publication` holds an immutable envelope and one unique `message_id: string` for the Node. `Delivery` holds one destination's state. One Publication has `1..*` Deliveries; each Delivery has exactly one Publication. The Delivery key is `(message_id, destination)`. Its fields are `expected_peer: bytes[16]`, `ack: AckMode`, `attempts: uint32`, `accepted_at: Instant`, `deadline: Instant`, `next_attempt: Instant`, `wrapper: bytes`, and `notify: set<uint64>`. Retries use a monotonic clock so a system-clock adjustment does not move their deadline.

The Node uses a hash map for Deliveries and one queue for new work. Capacity includes queued and waiting reliable Deliveries. The field limit is 128 Deliveries. Best-effort traffic uses a separate 64-item queue. A full reliable queue rejects new work. A full best-effort queue removes its oldest item and increases `dropped`. Durable queue storage remains part of D2.

3. Core algorithms

A1. Retry delivery until the requested receipt or expiry

1. Purpose. A1 keeps reliable publications during link outages. It prevents an early receipt from deleting data before the Hub commit. It implements US-01 and US-02 through I3, I4, I5, and I6.

2. Inputs and outputs. Inputs are `PublishRequest`, verified `StatusUpdate`, `now: Instant`, and configured limits. Section 5 gives exact wire types. Output is `Result<bool, ErrorDetail>`, zero or more signed packet sends, and `StatusEvent` values. Status values are `RECEIVED`, `STORED`, `EXPIRED`, or `FAILED`.

3. Procedure and complexity. The Node checks the request and encodes its signed wrapper. It rejects remote wrappers above 383 bytes. It rejects duplicate message IDs with different envelope bytes. It admits one Delivery per destination. A repeat of the same pending request reuses the Delivery without extending its deadline. The Node adds the caller to its notification set. The first attempt occurs immediately. The deadline is acceptance plus 3,600 seconds.

At each one-second tick, the Node checks expiry before sending. It expires Deliveries whose deadline is at or before now. It sends each remaining due wrapper once. It increases the attempt count and sets the next attempt to now plus 30 seconds. A send error keeps the Delivery. A delayed tick sends once; it does not send a burst of missed attempts. The interval is a minimum; the carrier can delay a send.

The Node checks each remote receipt's signature and expected peer. It resolves the destination from that peer's configured service. For `ACK_STORED`, `RECEIVED` does not remove the Delivery. A valid `STORED` removes only that destination's Delivery. For `ACK_RECEIVED`, a valid `RECEIVED` or `STORED` completes it. A remote `FAILED` or `EXPIRED` does not prove storage. The Node keeps an `ACK_STORED` Delivery until local expiry. It deletes a Publication only when it has no Deliveries.

Admission and receipt lookup take expected `O(1)` plus `O(B)` encoding and signature work. A tick takes `O(Q + D*B)`, where Q is pending Deliveries, D is due sends, and B is wrapper bytes. Memory takes `O(Q*B)` plus notification sets. The scan uses library containers; it does not require custom sorting.

Planning size	Estimated work and effect
One field Node	One message per 60 seconds gives at most 60 pending messages in one hour, with one destination. A tick checks about 60 entries.
100 times that load	A shared test Node holds 6,000 Deliveries. A tick checks 6,000 entries. Its test capacity is 8,192. This does not fit the field limit.
Storage estimate	At the 383-byte wrapper limit, 60 wrappers use about 23 KB; 6,000 use about 2.3 MB, before container overhead.

4. Why this algorithm. A simple scan suits the bounded field queue. A minimum heap reduces idle checks to `O(log Q)` updates but adds cancellation state. Stop-and-wait blocks other sensors during an outage. Five retries end after about 150 seconds and cannot cover US-01. A heap becomes an option if measured scheduler cost limits performance at 100 times load. Radio airtime can dominate these CPU costs.

5. Edge cases. An empty queue sends nothing. Equal retry times each get one send. Their order does not affect storage. Repeated receipts after completion do nothing. A wrong peer, bad signature, or unknown message does not remove data. Full capacity returns `QUEUE_FULL` without removal. At exactly the deadline, expiry wins. With several services on one peer, each destination uses a distinct message ID because v1 receipts omit destination. Node restart loses memory state; persistent storage remains part of D2.

A2. Check and commit a complete envelope once

1. Purpose. A2 converts a verified delivery into one stored row per sensor. It sends `STORED` only after a complete commit. It implements US-02, US-04, and US-05; AC-01.1 to AC-01.3; I6 and I7.

2. Inputs and outputs. Input is `DeliveryEvent{destination: string, message: PulseEnvelope, authenticated_sender: bytes[16]}` plus `receivedAtMs: int64 > 0`. Output is `StoreResult = STORED | DUPLICATE` or

`StoreError`. `StoreError` is `INVALID_ENVELOPE | UNKNOWN_SENSOR | UNSUPPORTED_ENCODING | INVALID_PAYLOAD | IDENTITY_MISMATCH | CONFLICTING_DUPLICATE | STORAGE_ERROR`. These are internal codes, not new Protobuf enum values. Success causes `AckRequest{message_id, status: STORED, detail: ""}`. Failure causes a log entry and no receipt.

3. Procedure and complexity. The Hub checks the envelope fields and sender binding. It checks each sensor against the Sensor table. It rejects repeated sensor IDs. The decoder registry selects a library parser for each encoding. If any sample fails validation, the Hub rejects the whole envelope.

The Hub encodes the parsed envelope with the pinned Java Protobuf codec. It stores this representation as `envelope_bytes`. This uses the same codec for every comparison; it does not compare arbitrary Protobuf field order on the network. The Hub starts one `BEGIN IMMEDIATE` transaction. It looks up `(origin_id, message_id)`. If the key exists, it compares stored bytes and sender with the new values. A match returns `DUPLICATE`. A difference causes rollback and returns `CONFLICTING_DUPLICATE`. A retry must keep sample order and payload bytes unchanged.

For a new key, the Hub inserts one Envelope and all its Readings. It commits the transaction, returns `STORED`, and sends the receipt. A failed commit rolls back all new rows. A failed receipt send leaves committed rows unchanged. The next matching retry returns `DUPLICATE` and sends the receipt again. The PK check inside the transaction handles concurrent writers. The transaction reports data conflicts rather than ignoring them.

Let S be samples, B be envelope bytes, R be stored Readings, and E be stored Envelopes. Checks take `O(B + S)` with a set for sensor IDs. Indexed inserts take `O(S log R + log E)`. Memory takes `O(B + S)`. Commit latency also depends on disk sync.

Planning size	Estimated work and effect
Initial farm	10 devices, one sample per minute: 14,400 Readings/day and 432,000 in 30 days. Average intake is 0.167 envelopes/second.
100 times that load	1,000 devices: 1,440,000 Readings/day and 43,200,000 in 30 days. Average intake is 16.67 envelopes/second.
Index and disk estimate	`log2(R)` rises from about 19 to 26. A planning allowance of 1 KB per sample gives about 432 MB or 43.2 GB.

The estimates use 10 devices with one sensor each. Allow extra disk space for indexes and WAL. The larger case needs a storage and commit benchmark; radio capacity is a separate limit.

4. Why this algorithm. A transaction prevents partial envelopes. A primary key and full content comparison give persistent duplicate checks. An in-memory set loses history after restart. A separate lookup followed by an unlocked insert races with another writer. A receipt before commit loses data after a crash. Generated Protobuf bytes avoid a custom hash or canonical JSON scheme.

5. Edge cases. The Hub rejects empty samples, unknown encodings, empty sensor IDs, negative times, and repeated sensor IDs. Equal times remain separate messages. Queries order results by time, then message ID. Unknown collection time stays zero. An identical retry returns `DUPLICATE`. Changed payload under the same key returns a conflict. Unapproved remote identities never reach storage. Disk full, busy timeout, and a failed commit send no receipt. A crash before commit leaves no new envelope; a crash after commit permits a matching retry.

4. Build-versus-reuse decisions

These choices use the D1 dependency snapshot [P3]. Rust versions are pinned in Cargo.lock and Java versions in gradle/libs.versions.toml. We build the delivery rules and reuse libraries for codecs, storage, and cryptography.

Piece	Build or reuse; license	Maturity, performance, and fit
Delivery lifecycle	Build; Pulse course license	Project-specific retry and receipt rules. Bounded scans suit 60 pending messages. Library containers provide the basic operations.

Piece	Build or reuse; license	Maturity, performance, and fit
Atomic storage and duplicate rules	Build; Pulse course license	The key, transaction order, and content conflict rule belong to Pulse. SQLite provides transaction recovery.
Decoder registry and sensor checks	Build; Pulse course license	US-04 needs a small encoding-to-decoder map. Each decoder defines its fields and units.
Local RPC and framing	Build the contract; Pulse course license	I3/I6 need a shared small frame, explicit errors, and Rust/Java vectors. Generated codecs encode the body.
Async I/O and timers	Reuse Tokio 1.53.1; MIT [W2]	Maintained runtime with documented support policy. Bounded channels and timers fit OS hosts. ESP32 runtime fit remains a D2 task.
Protobuf codecs	Reuse prost 0.13.5, Apache-2.0; protobuf-java 4.32.1, BSD-3-Clause [W3, W4]	Established schema tools produce compact binary messages. Shared vectors check language agreement. Generated bindings preserve stable field numbers.
Signature and verification	Reuse ed25519-dalek 3.0.0; BSD-3-Clause [W5]	Established library family; version 3 matches the SDK. The library signs exact bytes. D2 covers board measurements.
Network stack	Reuse reticulum-sdk 2.3.4; MIT [W6]	Small upstream project; direct LoRa support is experimental. Our dependency pins this version. It provides paths and encryption. Hardware and Python interoperation remain required checks.
Database engine and driver	Reuse SQLite, public domain; Xerial 3.49.1.0, Apache-2.0 and BSD-2-Clause notices [W7, W8]	Mature embedded database; maintained JDBC driver. One writer fits initial intake. Host disk speed determines FULL commit latency.
JSON decoding	Reuse Gson 2.13.1; Apache-2.0 [W10]	Maintenance mode is sufficient for JSON parsing. The decoder uses strict parsing and explicit field checks. Adding it to runtime is a D1 target; it is a test dependency today.

We retain dependency license notices. The smaller Reticulum SDK needs particular attention to version pinning and interoperability tests.

5. API contract

Transport, session, and common types

I3 and I6 use persistent TCP on `127.0.0.1:4401` by default. Configuration can select another loopback port from 1 to 65,535. One frame contains a 4-byte unsigned big-endian body length and one Protobuf `RpcEnvelope`. Body length is 1 to 1,048,576 bytes. The Node checks the length before allocating the body. A bad length closes the connection. These methods use protocol error codes, not HTTP status codes.

The common wrapper is `RpcEnvelope{protocol_version: uint32 = 1, request_id: uint64, op: Operation, body: oneof}`. A request uses a required ID from 1 to 2^64-1, unique among that connection's pending requests. A response echoes that ID. An event uses ID zero. `op` and `body` must agree. Proto3 defaults do not make an application-required value optional.

Type or field	Exact rule
`Name`	Required ASCII string, 1-64 characters, using letters, digits, _, or -. Applies to origin, sensor, service, and destination names. Service `node` is reserved. D1 uses service names, not arbitrary hashes.
`message_id`	Required canonical lowercase UUID v4 string, 36 characters. Retries keep it unchanged. Within one Node it identifies one immutable Publication.

Type or field	Exact rule
`PulseEnvelope`	`{envelope_version:uint32, message_id:string, origin_id:string, collected_at_ms:int64, samples:list<SensorSample>}`. Version is 1. Origin is Name. Time is optional, 0 when unset; otherwise 1 to 2^63-1 UTC milliseconds.
`SensorSample`	`{sensor_id:string, encoding:string, payload:bytes}`. Sensor is Name and is unique within the envelope. Encoding is required, 1-128 ASCII characters, and names an installed decoder.
Samples and payload	Required list with 1-16 samples. Each payload contains 1-262,144 bytes. The total local frame must fit the frame limit. Remote publication also requires its complete signed wrapper to fit 383 bytes.
`AckMode`	`ACK_NONE=0`, `ACK_RECEIVED=1`, `ACK_STORED=2`. Optional; default is `ACK_NONE`. Other values are invalid. Farm telemetry explicitly requests `ACK_STORED`.
`Response`	Success: `{ok:true, error:absent, health:absent}`; HEALTH adds HealthStatus. Failure: `{ok:false, error:{code:ErrorCode,message:string}, health:absent}`. The message explains the cause; callers use the code.

I5 carries `TransportPacket{payload:bytes, sender_identity:bytes[16], signature:bytes[64]}`. The payload contains `SignedPayload{ack:AckMode, body:data(bytes) | status(StatusUpdate)}`. Data bytes encode one PulseEnvelope. The signature covers the exact payload bytes. The configured public identity contains 64 bytes, shown as 128 hex characters. The 16-byte hash refers to that identity; it is not the full public key. Reticulum owns the route and hop limit [P3].

The 383-byte cap comes from the pinned SDK's encrypted single-packet limit. It includes the whole Pulse wrapper. A 1 MiB local frame does not imply a 1 MiB radio packet. Oversize remote data returns `INVALID_MESSAGE`; D2 can define fragmentation if required.

Key operations on C3

Method	Inputs, required values, and limits	Output and errors
`PUBLISH` / I3	Required `destination:Name`, `message:PulseEnvelope`. Optional `ack:AckMode=ACK_NONE`, `want_acceptance:bool=false`. Sender origin must match the local configured origin.	Success returns Response if acceptance is true. Failure always returns Response. `INVALID_MESSAGE`: bad fields, conflicting pending ID, unsupported enum, or oversize wrapper. `SERVICE_UNAVAILABLE`: no configured provider. `QUEUE_FULL`: reliable capacity is full. `UNAUTHORIZED`: origin binding fails.
`SUBSCRIBE` / I6	Required `service:Name`; it must name a service allowed for this local component. Repeat registration on the same connection has no extra effect. Only one active Hub consumer owns a storage service.	Success returns Response. Deliveries then use Event. `INVALID_MESSAGE`: invalid or reserved service. `SERVICE_UNAVAILABLE`: another consumer owns it. `UNAUTHORIZED`: the component cannot host it.
`ACK` / I6	Required `message_id:string` under the UUID rule; required `status:DeliveryStatus=STORED`. Optional `detail:string=""`, 0-256 UTF-8 bytes. The calling connection must own the service's pending storage delivery.	Success returns Response and causes a signed remote storage receipt. `INVALID_MESSAGE`: unknown waiter, non-STORED status, or bad fields. `UNAUTHORIZED`: another connection attempts the receipt. A lost remote receipt triggers a new delivery; its new waiter permits another ACK.
`HEALTH` / I3 or I6	Required empty `HealthRequest{}`; no parameters. Wrapper version and request ID rules still apply.	Success returns Response with HealthStatus. `INVALID_MESSAGE`: version or operation/body mismatch. A closed connection yields a client I/O error.

The implementation snapshot [P3] still needs the name limits, origin binding, and ACK ownership checks described here. Loopback restricts network access; it does not authenticate every local process. The first deployment uses one trusted service account and protected configuration files.

Key method inside C5

`ReadingStore.store(envelope:PulseEnvelope, authenticatedSender:bytes[16], receivedAtMs:int64)` returns `StoreResult = STORED | DUPLICATE`. All arguments are required. The limits match the common types. `receivedAtMs` is 1 to 2^63-1 UTC milliseconds. A2 defines the checks and transaction order. The method generates comparison bytes from the parsed envelope with the pinned Java Protobuf codec.

Internal error	Cause and caller action
`INVALID_ENVELOPE`	A field, count, time, or duplicate sensor fails a rule. The Hub logs the error and sends no receipt.
`UNKNOWN_SENSOR` / `UNSUPPORTED_ENCODING` / `INVALID_PAYLOAD`	A sensor is not configured, a decoder is absent, or its parser/check fails. The Hub rejects the whole envelope.
`IDENTITY_MISMATCH` / `CONFLICTING_DUPLICATE`	The identity binding fails, or the same key carries different bytes. The Hub keeps committed data and sends no receipt.
`STORAGE_ERROR`	SQLite returns busy timeout, disk full, or another transaction error. The Hub rolls back and sends no receipt. The origin retries until expiry.

The current Java method returns Outcome and throws SQLException. The D1 method adds the listed checks and internal errors. These errors stay inside C5; they do not add unassigned wire enum values.

Events, status binding, and error recovery

Output	Exact shape, units, and conditions
`DeliveryEvent` / I6	`{destination:string, message:PulseEnvelope, authenticated_sender:bytes}`. Destination is the local service hash. Sender is exactly 16 bytes for remote data or empty for local data. C5 accepts an empty sender only for a configured local origin.
`StatusUpdate` / I5	`{message_id:string, status:DeliveryStatus, detail:string}`. Detail has at most 256 UTF-8 bytes. A signed expected peer supplies it. Completion requires a matching peer and pending service.
`StatusEvent` / I3	`{message_id:string, status:DeliveryStatus, detail:string}`. `RECEIVED=1`, `STORED=2`, `EXPIRED=3`, `FAILED=4`. Zero is invalid.
`HealthStatus`	`{inventory_id:string, boot_id:string, uptime_seconds:uint64, component:string, queue_depth:uint32, dropped:uint64, last_contact_ms:int64}`. Inventory is Name; boot ID is UUID v4; component is `node`. Queue depth counts pending Deliveries, including waiting items. Dropped counts best-effort removals since boot. Contact is epoch milliseconds, zero before any verified peer contact.

For local deliveries, C5 resolves an empty sender to the configured local identity before storage. The schema always stores a 16-byte identity hash. Collection and receipt times remain distinct. Status events omit destination in v1. Therefore, the Edge assigns distinct IDs to different destinations when one peer provides several services. A later schema can add explicit destination binding.

Error code or failure	When it occurs	Caller response
`INVALID_MESSAGE=1`	A supported frame has invalid Protobuf, version, operation/body, or application fields.	The caller corrects the request before retrying. A malformed body gets request ID zero if no ID can be read.
`SERVICE_UNAVAILABLE=2`	A destination lacks a provider or the storage service has another active owner.	The caller retries when the configured service becomes available.
`QUEUE_FULL=3`	Pending reliable Deliveries reach capacity.	The caller retains the sample and retries admission later. The Node does not own a rejected sample.
`UNAUTHORIZED=4`	A readable local request fails an origin, service, or ACK ownership check.	The caller corrects its configuration or identity before retrying.
Invalid remote sender	Peer is unknown, signature is bad, or origin/service binding fails.	Receiver drops and logs the packet. It sends no trusted negative receipt. This implements AC-01.3 without replying to an untrusted sender.
Invalid length or closed stream	Zero/oversize frame, truncated stream, or I/O failure.	The client reconnects after 100 ms and doubles the delay to a 5,000 ms cap. It subscribes again. No event replay exists on the connection.

A valid `STORED` is the only success that proves Hub storage for `ACK_STORED`. Expiry means storage is unconfirmed; it does not prove that the Hub has no copy. A missing receipt after commit can produce this result. The Node reports the result so the technician can investigate.

Versioning. `protocol_version` versions local RPC; `envelope_version` versions data; both start at 1. Compatible changes add optional Protobuf fields and preserve existing field numbers. Changing a field type, required rule, receipt meaning, or supported identity width breaks the contract. Such a change requires a version increase and coordinated peer upgrade. D0's identity-width change needs no migration of deployed farm devices because D0 is a design baseline. Future deployed v1 changes require a migration. A new sensor with an installed decoder does not require a new envelope version.

Example request and response

The example shows Protobuf JSON notation for readability. The wire uses Protobuf bytes with the length prefix. Bytes use base64. JSON uses strings for 64-bit integer fields. The sample payload decodes to `{"moisture":0.42}`. Moisture uses a fraction, not a percentage.

```
{
  "protocolVersion": 1,
  "requestId": "17",
  "op": "PUBLISH",
  "publish": {
    "destination": "hub-main",
    "message": {
      "envelopeVersion": 1,
      "messageId": "11111111-2222-4333-8444-555555555555",
      "originId": "field-01",
      "collectedAtMs": "1788912000000",
      "samples": [{
        "sensorId": "soil-1",
        "encoding": "application/json",
        "payload": "eyJtb2lzdHVyZSI6MC40Mn0="
      }]
    },
    "ack": "ACK_STORED",
    "wantAcceptance": true
  }
}

{
  "protocolVersion": 1,
  "requestId": "17",
  "op": "RESPONSE",
  "response": { "ok": true }
}
```

After the Hub commits, the field Node sends this event to the Edge.

```
{
  "protocolVersion": 1,
  "requestId": "0",
  "op": "EVENT",
  "event": { "status": {
    "messageId": "11111111-2222-4333-8444-555555555555",
    "status": "STORED",
    "detail": ""
  } }
}
```

An admission failure instead returns `response:{ok:false,error:{code:QUEUE_FULL,message:"reliable outbound queue full"}}` under request ID 17. Acceptance and `STORED` remain separate results.

6. Technology choices with justification

Database: SQLite through Xerial JDBC

We chose SQLite because the Hub needs one local writer, transactions, and duplicate checks without another service to install. WAL supports concurrent reads. Xerial provides the Java driver [W1, W7, W8]. PostgreSQL would add setup and administration that the initial workload does not need.

Backend: Rust Node with Tokio; Java 21 Hub

Rust and Tokio provide bounded queues, timers, and asynchronous I/O for the Node. Java 21 gives the Hub a straightforward JDBC storage path. The JVM runs only on the Hub host. We kept these existing project choices rather than adding a web framework or rewriting the Node in another language.

Interface: command-line tools and logs

Technicians configure devices and inspect delivery faults through commands, logs, and HEALTH. The user stories do not require a dashboard, so we are not building one.

Messaging and hardware: Protobuf, Reticulum, and ESP32-S3

Protobuf defines the shared Rust/Java messages. Reticulum provides the carrier layer, starting over TCP before board testing. We use the existing ESP32-S3 and LoRa hardware. We chose this path over a central MQTT broker because field delivery must work without a separate IP service. D2 defines the LoRa packet limits and board recovery design.

Hosting: local Linux host with systemd

The prototype runs on an existing Linux host. systemd restarts the Node, Edge, and Hub independently. A local database keeps farm internet access out of the collection path. We use the $200 course budget and existing equipment [P4]; any additional purchase must fit that cap.

Project sources

- **P1:** `documents/sd1/Design_Diagrams/D0_High_Level_Design.md`, with its original figure sources. D0 provides the component and interface baseline.
- **P2:** `User_Stories.md`: US-01 to US-05, UC-01, and AC-01.1 to AC-01.3.
- **P3:** The sibling `monorepo`, commit `c845c4e`, including: `node/src/buffer.rs`, `node/src/node.rs`, `node/src/rpc.rs`, `hub/.../ReadingStore.java`, `HubMain.java`, `DecoderRegistry.java`, and `common/protocol/proto/pulse/v1/*.proto`. Dependency sources: `Cargo.toml`, `Cargo.lock`, `gradle/libs.versions.toml`, `MODULE.bazel`, and `LICENSE`.
- **P4:** `documents/sd1/assignments/assignment-3/Constraints_Essay.md`: $200 course budget and existing board inventory.
- **P5:** `docs/meetings/2026-09-11-first-advisor-meeting.md`: desktop-first plan and board tests.

External sources and license checks

Dependency documentation and license sources follow.

ID	Source and supported decision
W1	SQLite WAL: commit sync and single-writer behavior.
W2	Tokio: runtime support and MIT license.
W3	prost: generated Rust codecs and Apache-2.0 license.
W4	Protobuf: schema tools; license.
W5	ed25519-dalek: signature implementation. Pinned 3.0.0 manifest states BSD-3-Clause.
W6	Reticulum SDK: MIT terms and experimental direct LoRa support. Pinned 2.3.4 packet source defines the 383-byte limit.
W7	SQLite copyright: public-domain terms.
W8	Xerial JDBC: maintenance, supported hosts, and license files.
W10	Gson: maintenance mode and Apache-2.0 license.

Pulse - Detailed System Design (D2)

Assignment 7 - Detailed System Design, Part 2 (D2) | October 14, 2026

Team: Jacob Cohen, Sujal Choukse, Yatharth Bajaj. **Advisor:** Jeffrey Gundler.

1. Header, conventions, and D2 overview

Project title: Pulse - platform-independent IoT communication for resilient farm sensor networks.

Goal statement: Move every farm sensor reading from a field device to a trusted Hub over any available link, and keep each reading until the Hub stores it.

Pulse collects farm sensor readings, carries them across an intermittent link, and stores them at a local Hub. The field Node keeps admitted readings until the Hub confirms storage, rejects the envelope permanently, or the delivery deadline expires. A successful local publish means the Node has accepted responsibility; only STORED proves that the Hub committed the reading.

D0 named six components and nine interfaces. D1 detailed the Node's retry algorithm (A1), the Hub's storage algorithm (A2), and the shared RPC contract. D2 finishes the Sensor Driver, Edge, Transport Adapter, and Builder. It also adds durable delivery storage, Hub maintenance, and the authentication, notification, configuration, and logging rules.

D2 is the Assignment 7 update to the Assignment 4 design-diagram collection. D1 was Assignment 6; D0 keeps its original header.

Design basis. This document follows the D0, D1, user stories, constraints essay, and advisor record in the resources repository. D1's Reticulum-over-TCP baseline and identity format take precedence over D0's older UDP description. D0 is unchanged. D1 has been edited for clarity; its technical baseline is retained. Part 5 records the changes.

Conventions

Each figure includes the project title, goal, and its own legend. In Figure 1, boxes are database tables and labeled lines are foreign keys; PK and FK mean primary and foreign key. In Figures 2-4, rectangles are actions or states, grey boxes are entry or final outcomes, solid arrows are normal transitions, and red dashed arrows show failures. Arrow labels give the condition and, for failures, an EC reference. C1-C6 match D0; EC IDs point to part 3.

D0 component	Where the design is completed
C1 Sensor Driver	2.1: state, sampling, and driver contract
C2 Edge	2.2; Figure 2: admission and reconnect
C3 Node	D1 A1; 2.3; Figure 3: durable delivery
C4 Transport Adapter	2.4: carrier mapping and send/receive
C5 Hub	D1 A2; Figure 1; 2.5: storage and disk space
C6 Builder	2.6; Figure 4: validation and output files
Shared services	2.7: authentication, health, configuration, logs

Delivery window and implementation checks

A message has 3,600 seconds from Node admission to confirmed storage. That window includes the outage, queued radio time, the Hub commit, and the receipt. US-01 now states this explicitly. At the deadline, the Node reports EXPIRED if storage is still unconfirmed.

Before admission, the Edge holds one sample per sensor and pauses that sensor if the Node is full. Admitted samples survive a Node restart; unadmitted samples can be lost if the Edge restarts.

The first deployment uses one Hub, existing Linux hosts, and the $200 course budget. The ESP32 target adds a battery-backed RTC. Before field deployment, we need to verify flash recovery after power loss, RAM use, radio packet size and airtime, RTC operation, and the latency budgets in section 2.8. Loopback connections assume trusted local processes and protected key files.

2. Finished components, the D2 diagrams

Names follow D1: 1-64 ASCII letters, digits, `_`, or `-`. Envelopes use version 1, a lowercase UUID v4, 1-16 unique sensor IDs, and unchanged payload bytes on retries. The 1 MiB local frame limit and 262,144-byte per-sample limit do not override the smaller remote wrapper limit.

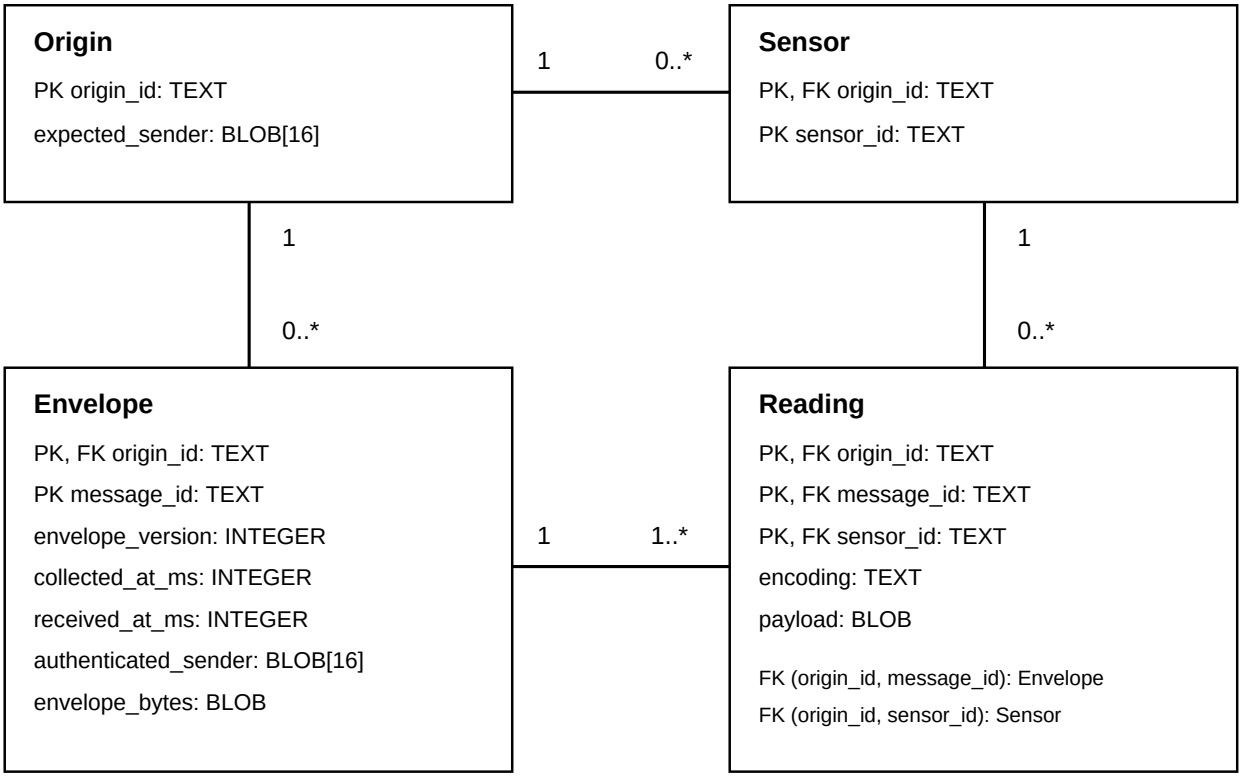
Pulse | Figure 1. Hub data model

Goal: move every farm reading to a trusted Hub and keep it until stored.

Legend: boxes = tables; lines = foreign keys; 1 / 0..* / 1..* = cardinalities.

Figure 1. C5 Hub: relational data model

Target D1 schema. PK = primary key; FK = foreign key.



A2 enforces one or more Readings per Envelope in one transaction. Lines show relationships, not message flow.

Figure 1 retains D1's Origin, Sensor, Envelope, and Reading entities. Figures 2-4 below detail the completed runtime and configuration behavior.

2.1 C1 Sensor Driver - Sujal Choukse

Data model. No persistent data. `SensorSpec` contains `id: Name`, `driver: Name`, and `interval_seconds: u64` in 1-86,400. Driver-specific settings include bus address, units, and valid range. A simulation driver holds a seed and tick counter; a hardware driver holds its bus handle.

Behavior. At each scheduled tick: open or reuse the bus, read with a five-second deadline, check the sensor's defined range, encode the reading, and return the bytes. A timeout, disconnected sensor, or invalid value returns an error; the Edge logs it and continues at the next tick. The driver must enforce the read deadline so a blocked bus cannot stop other sensors.

I1/I2 signature	Result and errors
`sample(&mut self) -> Result<Vec<u8>, DriverError>`	1-262,144 bytes, subject to the complete envelope limits. `Sample` includes timeout, bus failure, or invalid value.
`encoding(&self) -> &'static str`	1-128 ASCII characters, naming an installed Hub decoder.
`new(&SensorSpec) -> Result<Box<dyn SensorDriver>, DriverError>`	`Config` for unsupported settings; fail startup before sampling.
`registry() -> [(Name, factory)]`	Unique build-time driver names. `drivers` lists the same registry.

Each driver also declares `max_payload_bytes() -> usize`. The Builder uses this bound to size the complete signed packet for the selected radio profile. A driver that exceeds its declared bound is stopped and reported in HEALTH.

A new sensor needs a driver entry, a matching Hub decoder, and a configured Sensor row. It does not change the envelope protocol (US-04).

2.2 C2 Edge - Yatharth Bajaj

Data model. No persistent data. Each sensor has a timer, a state (`RUNNING`, `BACKPRESSURED`, or `STOPPED`), a last error code, and one pending sample. The sample keeps its original UUID, payload, and collection time through admission retries. This release has one Hub destination per field Node. Farm telemetry sets `ack = stored` and `want_acceptance = true` explicitly.

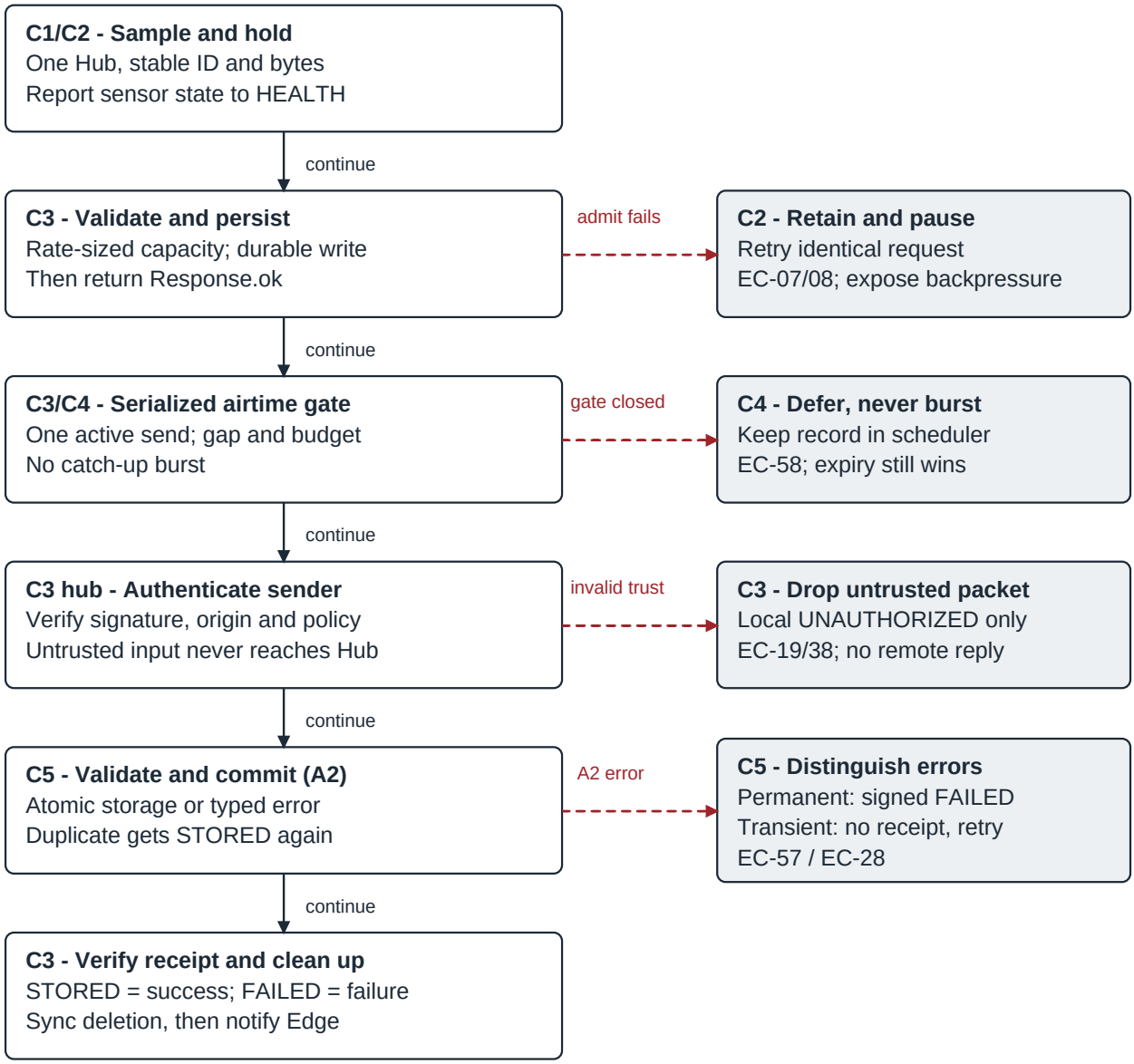
Behavior. At a sensor tick, sample only if that sensor has no pending sample. Publish the pending request and clear it only after `Response.ok`. On `QUEUE_FULL`, `SERVICE_UNAVAILABLE`, a five-second response timeout, or a broken connection, retain the identical request and mark the sensor backpressured. Retry on its next tick without replacing the sample. Skip missed ticks. A lost response can cause a duplicate publish; A1 reuses an identical pending Delivery without extending its deadline.

On `INVALID_MESSAGE` or `UNAUTHORIZED`, stop that sensor and report its ID and error in both the log and HEALTH. Other sensors continue. Reconnect after 100 ms, doubling to a five-second cap. A Node restart recovers admitted samples; an Edge restart can lose a sample that has not been admitted. The capacity rule in section 2.3 keeps the planned one-hour outage from filling the reference queue.

Pulse | Figure 2. Sampling to confirmed storage

Goal: move every farm reading to a trusted Hub and keep it until stored.

Legend: boxes = actions/states; solid arrows = normal; red dashed = failure.



All LoRa data and receipts share the radio gate; complete radio payload <=255 bytes.

Missing receipts retry safely. A trusted permanent rejection terminates as failure.

Expiry means storage is unconfirmed; it does not prove the Hub has no row (EC-13).

I3 interface	Result and errors
`pulse-edge run --config edge.toml`	Exit 0 on orderly shutdown; exit 1 for an unreadable or invalid configuration.
`pulse-edge drivers`	Print unique included drivers; exit 0.

I3 interface	Result and errors
`RpcClient::publish(request)`	Wait for Response. Timeout/I/O failure means acceptance is uncertain; retry identical bytes and ID.
`REPORT_SENSOR_HEALTH`	`{sensors:[{sensor_id:Name,state:SensorState,error_code:string(0-64 ASCII characters)}]}`. Authorized Edge connection only, at startup, on state changes, and every five seconds; at most 16 sensors. Invalid names/state/count return `INVALID_MESSAGE`; wrong connection returns `UNAUTHORIZED`.
`StatusEvent`	Existing message ID, status, and detail. Log STORED, EXPIRED, or FAILED. A FAILED report never means the sample was stored.

2.3 C3 Node - Jacob Cohen

D1 A1 owns validation, retry, receipt verification, and expiry. D2 adds durable acceptance, workload-based capacity, and paced sending.

Data model. Each record holds version 1, message ID, destination, expected peer hash (16 bytes), acknowledgement mode, accepted time and deadline (UTC milliseconds), immutable signed wrapper, and checksum. The key remains `(message_id, destination)`, although the release permits one configured Hub. Refuse a pending-capacity reduction below the number of recovered records; operator cleanup must finish first. Records are at most 1 KiB. Reliable work is separate from D1's volatile best-effort queue.

Capacity rule. Let E be the configured expiry (default 3,600 seconds) and I(s) each enabled sensor's sample interval. The Builder requires `C >= sum(ceil(E / I(s)) + 1)` for the one-Hub profile. The extra record per sensor covers a boundary tick. For three sensors at 60 seconds, the minimum is `3 x 61 = 183`; the reference capacity is 256. The outage produces about 180 records, leaving 76 slots at that point. Faster sampling requires a larger validated capacity or configuration rejection. The queue does not silently remain at 128.

Platform	Storage and recovery design
Linux	One record per file in `/var/lib/pulse/spool`, named by message ID and destination. Exclusive directory lock; temporary write, file sync, rename, and directory sync before success.
ESP32	A 2 MiB LittleFS partition and 256-record reference limit. A single writer uses temporary write/rename/flush; check free space before every admission. Reserve room for metadata, quarantine, and recovery. Do not enable reliable deployment until power-loss tests prove persistence on the selected filesystem/board.

256 records use at most 256 KiB before filesystem overhead. Queue indexes, task stacks, codecs, and carrier buffers must fit the board RAM. Increasing capacity requires checking both flash and RAM use.

Admission and recovery. Validate, check capacity and the carrier-specific packet budget, persist, then return success. At startup, lock the store, remove abandoned temporary files, validate committed records, quarantine corruption, and resume unexpired work with its original deadline. Failed persistence or recovery returns `SERVICE_UNAVAILABLE`, never success. Cleanup deletes and syncs the durable record before freeing capacity and emitting a final event. Failed deletion retains the record and retries cleanup; a crash may replay a safe duplicate.

Send scheduling. A1 marks a record eligible; it does not send all due records immediately. One scheduler per physical radio selects the earliest-deadline eligible record, breaking ties round-robin across peers. It allows one

transmission at a time. The next send starts only after the previous transmission ends, a minimum one-second inter-frame gap (or a larger profile guard), and the carrier's regulatory/airtime gate. Receipt packets use the same serialized gate and a bounded priority queue. No catch-up burst bypasses it.

After a send, the record becomes eligible again after 30 seconds. The radio gate spaces out due records. A gate deferral does not count as an attempt. Check expiry both at selection and immediately before transmission. At restart, eligible records re-enter this scheduler rather than sending in a loop. EDF favors records nearest expiry but cannot guarantee delivery when radio capacity is insufficient.

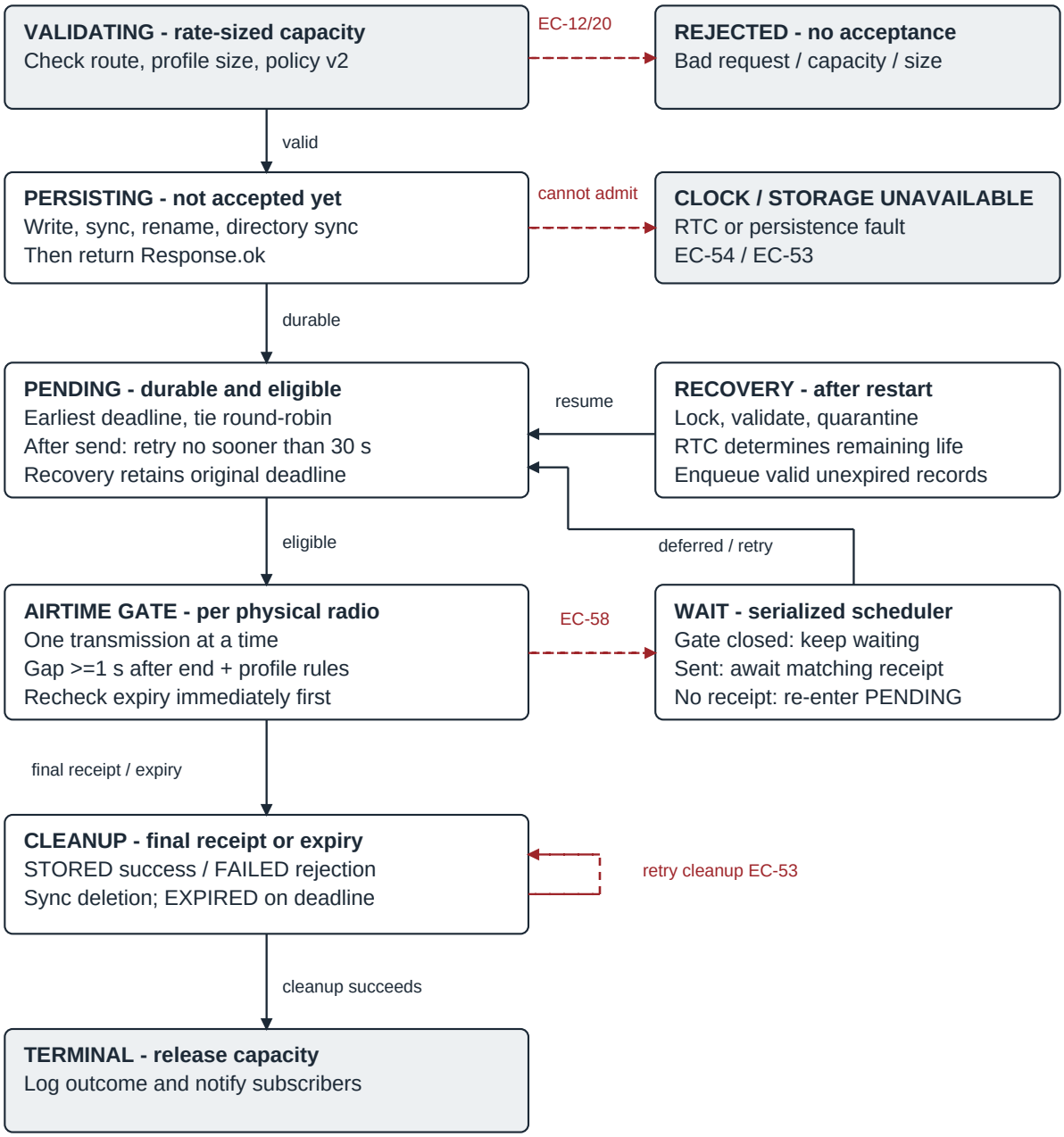
Receipt policy. A valid expected-peer STORED confirms storage. A valid expected-peer FAILED with a permanent rejection reason terminates that Delivery as failure; it never counts as successful storage. RECEIVED is insufficient for ACK_STORED. Missing receipts, broken sessions, and transient storage errors retain the Delivery until retry or expiry. This replaces D1's rule of ignoring remote FAILED for permanently invalid content.

Clock source. Linux uses its maintained system clock. The ESP32 field target uses a battery-backed DS3231 RTC on I2C, initialized in UTC from the Hub clock by the technician during bench provisioning. Check the RTC's oscillator-stop flag and plausible time at boot. The backup battery preserves time without internet or Hub contact. If the RTC is invalid, missing, or behind a stored acceptance time, hold replay and reliable admission until it is serviced; HEALTH reports the clock fault. Once boot time is trusted, a monotonic timer controls deadlines during that boot. Arbitrary clock changes across boots remain an operational fault.

Pulse | Figure 3. Durable Delivery and paced sending

Goal: move every farm reading to a trusted Hub and keep it until stored.

Legend: boxes = actions/states; solid arrows = normal; red dashed = failure.



A terminal FAILED is signed by the expected peer; it never proves storage (EC-57/61).

Interface. PUBLISH, SUBSCRIBE, ACK, HEALTH, and the health-report operation use local `protocol_version = 2`. Signed data/status payloads include `delivery_policy_version = 2`, checked before processing; the signature covers this field. Add it as a new Protobuf field without reusing an existing tag. A v1 Node must be upgraded with its peer; mixed versions fail compatibility validation. Envelope version and storage keys stay at 1. This coordinated version change prevents an old sender from ignoring terminal rejection semantics.

HEALTH adds `sensor_states`, `edge_connected`, `clock_state`, and optional `hub_storage` containing the last storage report. Sensor states returned by HEALTH also include STALE. A field Node without a local Hub returns no hub_storage report. Sensor state includes the reporting connection's last update time. A disconnected Edge or no heartbeat for 15 seconds marks its sensor information STALE, rather than healthy. The Node retains the most recent stopped-sensor reason for diagnosis. Internal Spool methods return typed storage errors; QUEUE_FULL means logical capacity, while persistence/clock/lock faults return SERVICE_UNAVAILABLE.

2.4 C4 Transport Adapter - Jacob Cohen

Data model. No persistent data. The adapter holds its carrier, a bounded 256-packet outbound queue, a peer-address map, and the radio scheduler's next-allowed-send time. Reticulum over TCP remains the desktop baseline. A separate UDP adapter and multiple-Hub routing are removed from this release.

Behavior. Resolve a peer, size-check the complete encoded packet, then queue it. For LoRa, the serialized scheduler in section 2.3 owns every data and receipt transmission. Decode inbound packets and pass them to the Node; a carrier address is diagnostic text, never authenticated identity. The Node verifies signatures and origin/service bindings. Reticulum owns routing and hop limits; Pulse adds no TTL.

Packet budget. D1's 383-byte Pulse-wrapper allowance applies only to the TCP baseline. The SX1262 LoRa profile uses at most 255 radio payload bytes per frame. Its Pulse-wrapper budget is `L = 255 - H`, where H includes the carrier header, encryption, and worst-case padding from the pinned encoder. Admission checks both `wrapper_length <= L` and `encoded_radio_payload_length <= 255`. The 383-byte TCP allowance does not apply to LoRa. Do not enable a profile that cannot supply a reliable bound. The selected profile manifest supplies the tested encoder revision, H, minimum guard/off-time, and measured usable data-send rate after reserving receipts/control traffic; a missing profile budget prevents LoRa approval. The Builder encodes each driver's declared maximum-size sample and every maximum-size receipt (including FAILED) with the configured names, signature, and version fields before approving that deployment. It also checks that complete sensor-health reports fit the target's local-frame limit. Over-budget samples require a smaller driver encoding; this release does not support fragmentation or long-packet mode.

I4 contract	Result and errors
`spawn(inbound)`	`Result<TransportHandle, TransportError>`; Bind or Unavailable stops startup.
`packet_budget(profile)`	Returns Pulse-wrapper limit L and maximum encoded frame length. Unsupported/unbounded profiles return Unavailable.
`send(peer_id, packet)`	UnknownPeer, Oversize, Busy, or Closed. Success means local queueing, not remote delivery.
`IngressPacket{source,packet}`	Diagnostic source and signed packet. Node verifies the sender.

Busy/Closed/I/O failure keeps reliable work. A permanent bad route or oversized wrapper fails admission; if found later, terminate as FAILED after durable cleanup. Every radio send passes the airtime gate, including retry and rejection receipts.

2.5 C5 Hub - Yatharth Bajaj

Data model. Figure 1 and D1 A2 retain Origin, Sensor, Envelope, and Reading. The Hub validates an entire envelope, compares duplicates, and commits atomically. STORED is sent only after commit. Identical retries return DUPLICATE and get another STORED receipt.

Permanent rejection. If a trusted, authorized delivery has a valid message ID but an invalid envelope, unknown sensor, unsupported decoder, invalid payload, or conflicting duplicate, A2 rolls back and the Hub sends `AckRequest{message_id,status:FAILED,detail:<reason>}`. Only the local connection owning that pending delivery may request it. The hub Node sends the signed FAILED status to the origin. The origin verifies expected peer, version, and ID, then removes that Delivery as failure and reports it. No new REJECTED enum is added. Untrusted senders still get no reply. An unreadable/unusable message ID cannot be correlated, so it is dropped and logged; the origin eventually expires. Storage busy, disk-full, and failed-commit errors are transient: no FAILED or STORED is sent, so retry continues.

Retention and disk monitoring. Retain every Envelope and its duplicate key for at least 3,600 seconds after receipt. There is no automatic deletion or backup service in the required release. An operator can remove older rows transactionally, deleting child Readings before Envelopes. The Hub checks free space once a minute: warn below 10% and report critical below 5%. Log a threshold change once, and show the current level through `REPORT_STORAGE_HEALTH` and HEALTH. Mark the report stale after two missed updates. Storage continues while writes succeed; an actual write failure rolls back without acknowledgement. An operator must free space or archive older readings.

Sizing basis. D1's 432 MB estimate used 10 field devices with one sensor each, one sample per minute, 30 days, and 1 KB/sample. The revised three-sensor reference case has 30 sensors total: 43,200 samples/day, 1,296,000 in 30 days, about 1.296 GB before WAL, indexes, filesystem overhead, and free-space reserve. This estimate covers stored readings. WAL, indexes, and free-space reserve add to it; radio capacity is checked separately.

I6/I7 interface. `ReadingStore.store(envelope, authenticatedSender[16], receivedAtMs)` returns STORED or DUPLICATE, or D1's typed validation/storage error. In local protocol v2, ACK accepts STORED or FAILED from the delivery owner only. FAILED detail is one permanent A2 error-code name, at most 32 ASCII bytes, never free-form prose; STORAGE_ERROR is not permitted as a terminal rejection. `REPORT_STORAGE_HEALTH{free_bytes:optional<uint64>,total_bytes:optional<uint64>,severity:OK|WARN|CRITICAL|UNKNOWN}` is restricted to the subscribed Hub connection; require both byte counts for OK/WARN/CRITICAL, reject free > total or an invalid severity, and allow absent counts only for UNKNOWN after a measurement failure. These operations return INVALID_MESSAGE or UNAUTHORIZED on invalid input/ownership.

2.6 C6 Builder - Sujal Choukse

Data model. No persistent runtime state. Input is a version-1 TOML device configuration; outputs are `node.toml`, `edge.toml`, and `hub.flags` in one versioned output directory. Private keys are never embedded in these files. Device configuration declares `power_role = battery|powered`; battery targets must disable carrier forwarding. A LoRa build requires a supported frequency/power profile, its measured encoded-packet budget, and the serialized airtime gate. Reject a profile whose admitted sample rate equals or exceeds its usable data-send rate after receipt/control traffic is reserved.

Input area	Validation rules
Node	Valid `Name`; key-file path on OS hosts; positive expiry/retry limits; queue capacity at least the sensor-derived minimum; spool path and RAM/flash fit. ESP32 requires `clock.source = ds3231` and its I2C configuration.
Carrier and peers	Supported carrier and parseable addresses; unique peer IDs; public identities are 64 bytes encoded as 128 hex characters, per D1 and AC-02.1. Every route references a known peer. No Pulse TTL field.
Local API	Loopback TCP on OS hosts, port 1-65,535; bounded local queue on ESP32. Reject remote bind addresses.

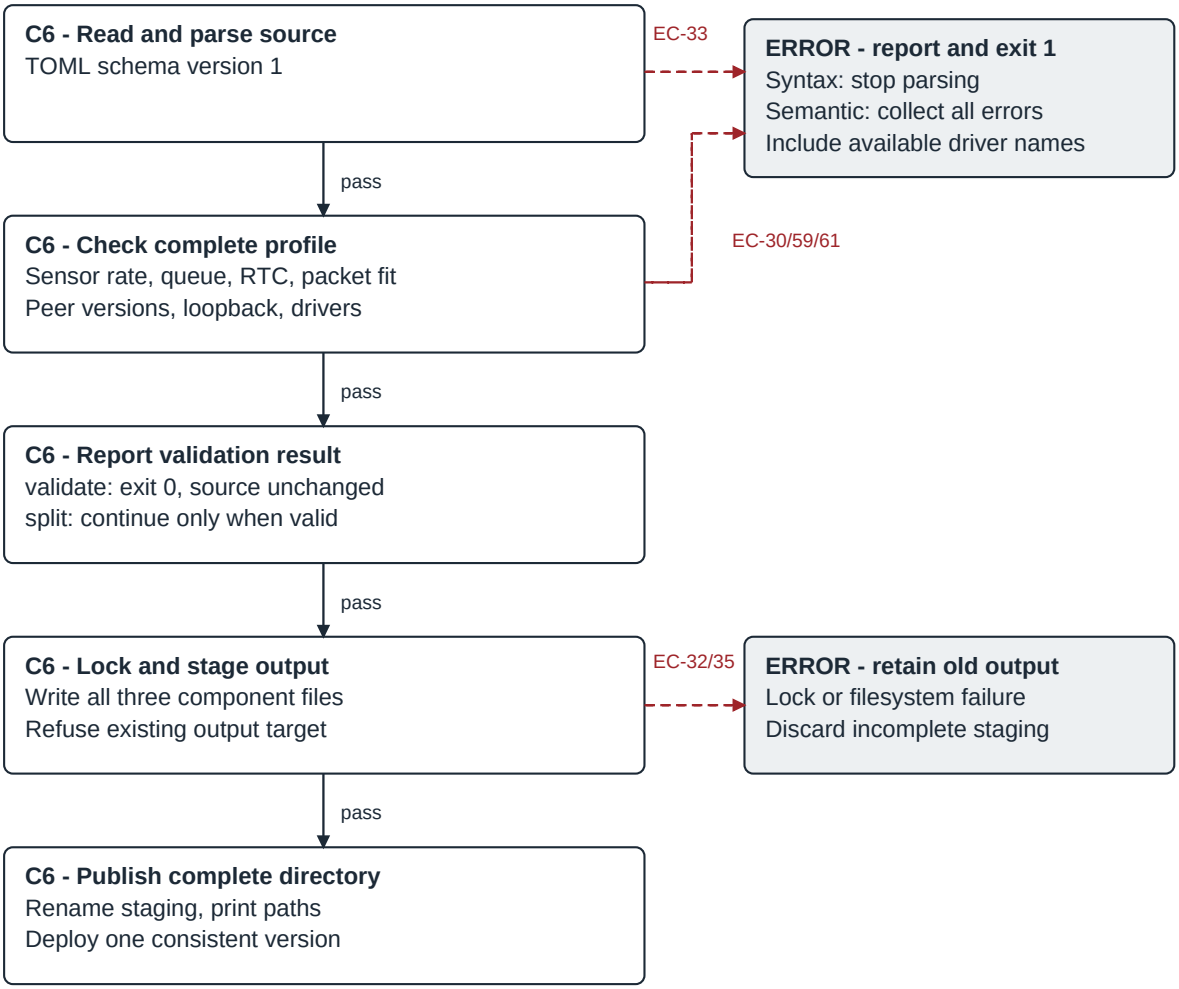
Input area	Validation rules
Edge	If enabled, 1-16 sensors and exactly one Hub destination; unique sensor IDs; registered drivers; interval 1-86,400 seconds; valid acknowledgement mode. Farm telemetry requires `stored` and acceptance replies.
Hub and platform	Hub runs on an OS host, not ESP32. Match the selected platform, schema version, and supported carrier build. Reject unknown fields rather than silently ignore typos.

Behavior. Parse the source file, then collect all independent semantic errors, including available driver names for an unknown driver. A syntax error stops semantic validation because no usable document exists. Validation never modifies the input. For `split`, first validate, then acquire an output lock and write all files into a new staging directory. Publish that directory by rename only when every file is complete. Refuse an existing target directory or concurrent writer. A failed run leaves the previous deployment intact; deployment selects one complete directory while services are stopped.

Pulse | Figure 4. Validate and publish configuration

Goal: move every farm reading to a trusted Hub and keep it until stored.

Legend: boxes = actions/states; solid arrows = normal; red dashed = failure.



Approve a deployment only when encoded packet, memory and arrival-rate budgets fit.

I8/I9 command	Output and failure behavior
`pulse-builder validate --config <file> --platform <target>`	Report valid and exit 0, or report errors and exit 1. Target is linux, windows, or esp32.
`pulse-builder split --config <file> --platform <target> --out <new-dir>`	Same validation; publish one complete output directory. Exit 1 for validation, lock, or filesystem failure.
`pulse-builder drivers`	Print each included driver exactly once. Build the Builder and Edge from the same source revision.

`hub.flags` remains a launch argument list for `--connect`, `--service`, and `--db`. The launcher passes arguments directly, without shell evaluation; paths containing spaces stay one argument. Build orchestration commands are outside the runtime design. I9 keeps the three output types but gains a directory-level consistency rule.

2.7 Shared services

Service and owner	Data model	Behavior	Interface and errors
Authentication - Jacob	Persistent private identity in a protected file (mode 0600 on Linux, owner-only access elsewhere); trust table maps a 16-byte identity hash to the 64-byte public identity and permitted origins/services. Hub Origin stores the expected 16-byte hash.	Use the pinned identity/signature library. Resolve the hash through the trust table, verify the signature over exact signed bytes, then check origin/service binding. Drop unknown or invalid senders without a remote reply. Log the local reason `UNAUTHORIZED`.	I5 carries the 16-byte hash and signature, as in D1; I6 carries the authenticated hash. Missing or malformed identity files stop startup. Never substitute a raw 32-byte key for the configured public identity.
Notifications - Yatharth	No persistent data. Per-Delivery subscribers and a 256-event local channel.	Emit final status after durable cleanup. Drop an event if its channel is full and count/log the loss; never drop delivery data to make room for notifications. A disconnected client resubscribes; historical events are not replayed.	Version-2 `StatusEvent` and extended `HEALTH`. Unknown event enums are logged and ignored. This release has no external alert service.
Configuration - Sujal	Persistent versioned configuration files, separate key material, and compiled driver registry.	Load once at startup, validate consistently with the Builder, and refuse unsupported combinations. No hot reload. Use systemd on Linux with a five-second restart delay and start-rate limiting; configuration errors require repair. Other hosts use their own supervisor.	I8/I9. Node uses `local_api.bind`, Edge uses `local_api.connect`, and Hub uses `--connect`; default loopback endpoint is `127.0.0.1:4401`. Unreadable files or wrong schema stop startup.
Logging - Jacob	No Pulse-owned log database. Bounded in-memory log queue; OS journal on Linux, bounded serial output on ESP32.	Record level, component, message ID, origin, and error code. Exclude payloads and private keys. Log the first destination failure, then a recovery summary with the retry count. Full queues drop log lines and increment a counter rather than block delivery.	Internal structured logging calls; technician reads the journal/serial output. Rust and Java log levels are configured separately. No journal on ESP32.

ESP32 local interface. Edge and Node run as tasks in one firmware image. I3 uses the same typed request/response/event semantics through two bounded queues: 16 requests and 16 replies/events. Each frame is at most 1 KiB on this target; oversize requests return `INVALID_MESSAGE` before allocation. A full request queue keeps the Edge's pending request for retry. Each request keeps its ID until a reply or five-second timeout. No local TCP server or Java Hub runs on the board. Health-report requests share those bounded queues.

2.8 Resource and latency budgets

For the first deployment, we use three sensors per field Node, 60-second sampling, one Hub, 256 durable records, and a 3,600-second admission deadline.

Budget	Limit and overload behavior
Sampling	Five-second driver deadline; skip missed ticks. One held sample per sensor; no unbounded Edge queue.
Local admission	p95 under one second at reference load; caller timeout five seconds. Durable write must finish before success; a timeout retries identical bytes.
Hub commit	p95 under one second at desktop reference intake (10 Nodes, 30 sensors, 0.5 samples/s); busy timeout five seconds, then rollback and retry.
Queue/flash	Three sensors produce about 180 one-hour records; minimum capacity 183, configured 256. At most 256 KiB record bytes plus index and filesystem overhead. Increase capacity only after RAM/flash fit validation.
Radio	One active transmission, at least one second after end-of-transmission plus any larger carrier guard/off-time. Thirty seconds is a minimum retry interval, not a promise to send the whole backlog every 30 seconds.
Recovery drain	If B records are waiting, arrival rate is lambda and measured usable data rate is mu, drainage needs at least B/(mu-lambda) seconds when mu > lambda. If mu <= lambda, backlog cannot drain; reject that deployment profile or lower sampling. Deadline expiry still wins if draining takes too long.
Health	Edge heartbeat five seconds, stale at 15; Hub disk poll once a minute, stale after two missed updates. Slow reporting cannot stall delivery; unavailable status is visible rather than assumed healthy.

3. Edge cases and error handling

B = boundary values; T = timing/concurrency; I = bad/missing input; X = external dependency. Every component, including shared services, has all four categories. Recoverable means operation can resume after the named recovery; it does not promise that an expired or invalid sample can be recovered.

ID	Component	Category	Edge case	Design decision
EC-01	C1 Driver	B	Value outside the sensor range.	Reject the sample. Handler: `SensorDriver::sample`. Recoverable at the next valid read.
EC-02	C1 Driver	T	Bus read exceeds five seconds.	Return timeout; skip missed ticks. Handler: driver deadline. Recoverable on next tick.
EC-03	C1 Driver	I	Unknown driver or missing settings.	Fail startup with `Config`. Handler: registry factory. Recoverable after configuration/build repair.
EC-04	C1 Driver	X	Sensor disconnected.	Log and skip sample. Handler: bus driver. Recoverable after reconnection.
EC-05	C2 Edge	B	Interval is 0 or above 86,400 s.	Reject; no silent clamping. Handler: `validate_edge`. Recoverable after correction.
EC-06	C2 Edge	B	Local payload/frame or remote wrapper too large.	`INVALID_MESSAGE`; stop the affected task and report size. Handler: admission and sensor loop. Sample is not recoverable without re-encoding.
EC-07	C2 Edge	T	Node queue is full.	Keep the pending request and pause that sensor; retry next tick. Handler: sensor loop. Recoverable when capacity frees.
EC-08	C2 Edge	T	Connection closes or acceptance times out.	Keep identical IDs/bytes; reconnect and retry. Handler: RPC client. Recoverable; acceptance may already have happened.
EC-09	C2 Edge	I	No destination, invalid origin, or disabled Edge.	Exit 1. Handler: `validate_edge`. Recoverable after configuration repair.
EC-10	C2 Edge	X	Collection clock is unset.	Use collection time 0; never invent a timestamp. Handler: sample clock adapter. Recoverable after clock repair; reliable admission also obeys EC-54.
EC-11	C2 Edge	X	Node unavailable.	Retain a pending sample; connect with capped backoff. Handler: session loop. Recoverable when Node returns; sampling pauses.
EC-12	C3 Node	B	Reliable queue reaches configured C (256 in the reference case).	Return QUEUE_FULL; evict nothing. Handler: A1 admission; Builder enforces C >= sensor-derived minimum. Recoverable after completion/expiry.
EC-13	C3 Node	B	Receipt/send competes with exact deadline.	Expiry wins at or after deadline. Handler: A1 serialized state transition. Not recoverable for that Delivery; storage may still exist.

ID	Component	Category	Edge case	Design decision
EC-14	C3 Node	T	Receipt arrives after expiry.	Ignore; do not recreate state. Handler: A1 receipt path. No retry needed.
EC-15	C3 Node	T	Crash during admission or cleanup.	Replay committed records; discard temporary ones; duplicates are safe. Handler: `Spool::recover`. Recoverable for valid durable records.
EC-16	C3 Node	I	Frame is empty, oversized, or truncated.	Reject/close session before oversized allocation. Handler: frame decoder. Recoverable by reconnecting with a valid frame.
EC-17	C3 Node	I	Pending ID reused with changed content.	`INVALID_MESSAGE`; preserve first publication. Handler: A1 admission. Conflicting request is not recoverable.
EC-18	C3 Node	X	Carrier busy, closed, or send fails.	Keep Delivery for a later allowed send. Handler: A1 send path. Recoverable before expiry.
EC-19	C3 Node	X	Remote sender is not trusted.	Drop and log local `UNAUTHORIZED`; no reply. Handler: ingress verifier. Recoverable only after approved provisioning.
EC-20	C4 Adapter	B	Wrapper exceeds the profile budget, or encoded LoRa payload exceeds 255 bytes.	Reject as Oversize before admission/send. Handler: packet-budget encoder. Not recoverable without smaller encoding/profile repair.
EC-21	C4 Adapter	T	256-packet outbound queue full.	Return `Busy`, no unbounded wait. Handler: `send`. Recoverable on a later A1 retry.
EC-22	C4 Adapter	I	Malformed packet or unknown peer ID.	Drop malformed input; return `UnknownPeer` for a bad send route. Handler: decoder/peer resolver. Recoverable after valid input/configuration.
EC-23	C4 Adapter	X	Carrier cannot bind or connect.	Return startup error; supervisor retries with a limit. Handler: `spawn`. Recoverable when dependency returns.
EC-24	C4 Adapter	X	Selected LoRa backend unavailable.	Reject startup; never silently switch carriers. Handler: carrier factory. Recoverable after supported build/hardware is installed.
EC-25	C5 Hub	B	Trusted envelope has 0 or 17 samples.	Reject whole envelope; signed FAILED when ID and ownership are valid. Handler: A2; 16 is valid. Invalid content is not recoverable.
EC-26	C5 Hub	T	Concurrent identical retries.	Serialize transaction; return duplicate and resend receipt. Handler: `ReadingStore.store`. Recoverable automatically.
EC-27	C5 Hub	I	Trusted sender supplies unknown sensor/decoder or invalid payload.	Roll back, return permanent reason through signed FAILED. Handler: A2 and owned ACK. Data must be corrected before a new publication.

ID	Component	Category	Edge case	Design decision
EC-28	C5 Hub	X	SQLite busy beyond 5 s or disk full.	Roll back, log `STORAGE_ERROR`, no receipt. Handler: storage transaction. Recoverable before expiry if storage returns.
EC-29	C5 Hub	X	Local Node session breaks.	Reconnect and subscribe; origin retries missing receipts. Handler: Hub session loop. Recoverable before expiry.
EC-30	C6 Builder	B	Enabled Edge has zero sensors.	Report semantic error, exit 1. Handler: configuration validator. Recoverable after edit.
EC-31	C6 Builder	B	Name empty, 65 characters, or non-ASCII.	Reject under shared `Name` rule. Handler: validator. Recoverable after edit.
EC-32	C6 Builder	T	Concurrent split or existing target.	Fail second writer; publish only a complete new directory. Handler: split lock and staging. Recoverable with a fresh target.
EC-33	C6 Builder	I	Invalid syntax, unknown field, version, or driver.	Exit 1; list semantic errors when parsing succeeds; show driver list. Handler: parser/validator. Recoverable after edit; source unchanged.
EC-34	C6 Builder	X	Builder and Edge have different registries.	Reject deployment mismatch; compare revision and device driver list. Handler: deployment check. Recoverable by rebuilding together.
EC-35	C6 Builder	X	Output path unwritable or disk full.	Exit 1; remove staging when possible; retain old output. Handler: split. Recoverable after filesystem repair.
EC-36	Authentication	B	Public identity is not 64 bytes.	Reject configuration (128 hex characters required). Handler: identity parser. Recoverable with valid provisioning.
EC-37	Authentication	T	Key rotated with pending Deliveries.	Drain or explicitly expire pending work before coordinated rotation. Old expected peers remain fixed per record. Handler: provisioning procedure. No automatic recovery for abandoned records.
EC-38	Authentication	I	Bad signature or unauthorized origin/service.	Drop, local `UNAUTHORIZED`, no remote reply. Handler: ingress verifier. Invalid packet is not recoverable.
EC-39	Authentication	X	Private identity file missing/unreadable.	Fail startup; never generate a replacement silently. Handler: identity loader. Recoverable after restoring the approved identity.
EC-40	Authentication	X	Signature library fails to initialize.	Fail startup; never disable verification. Handler: identity initialization. Recoverable after build/platform repair.
EC-41	Notifications	B	Event channel full.	Drop event, increment lost-event counter. Handler: event dispatcher. Delivery data remains recoverable; historical event does not.
EC-42	Notifications	T	Event arrives after Edge restart.	Log known status; do not alter admission state. Handler: Edge event loop. No recovery needed.

ID	Component	Category	Edge case	Design decision
EC-43	Notifications	I	Unknown/zero status.	Log and ignore. Handler: event decoder. Recoverable with compatible peer software.
EC-44	Notifications	X	Subscriber connection lost.	Discard that subscription; allow resubscribe; no replay promise. Handler: RPC session cleanup. Future events recoverable.
EC-45	Configuration	B	Port is 0 or above 65,535.	Explicitly reject, even if port 0 parses. Handler: address validator. Recoverable after edit.
EC-46	Configuration	T	File edited during execution.	Running snapshot remains active until restart. Handler: startup loader. Recoverable by controlled restart.
EC-47	Configuration	I	Wrong component file/schema.	Fail startup with the missing section/version. Handler: component loader. Recoverable after selecting correct file.
EC-48	Configuration	X	Config file unavailable or permission denied.	Fail startup with path and OS error. Handler: file loader. Recoverable after filesystem repair.
EC-49	Logging	B	Outage floods warnings.	Log the first failure and a recovery summary; count retries without logging each one. Handler: per-destination rate limiter. Recoverable automatically.
EC-50	Logging	T	Slow log sink or full log queue.	Drop log line, count loss; never block delivery. Handler: bounded logger. Lost lines are not recoverable.
EC-51	Logging	I	Payload or private-key field submitted.	Exclude it through structured field allowlist and review. Handler: logging wrapper. Recoverable by correcting the call site.
EC-52	Logging	X	Journal/serial sink unavailable or disk full.	Keep bounded logging; drop on overflow. Handler: log writer. Recoverable when sink returns; lost lines remain lost.
EC-53	C3 Node	X	Spool write, sync, delete, or quarantine fails.	Reject admission with `SERVICE_UNAVAILABLE`; retain existing records; retry cleanup. Handler: Spool. Recoverable after storage repair.
EC-54	C3 Node	X	DS3231 missing, oscillator-stop flag set, or time behind stored acceptance.	Pause reliable admission/replay and expose clock fault; technician provisions/restores RTC from Hub UTC. Handler: RTC boot gate. Recoverable after service.
EC-55	C5 Hub	T	Disk monitor overlaps storage or operator cleanup.	Read free space without holding the storage transaction; cleanup respects minimum retention. Handler: maintenance monitor. Recoverable automatically.
EC-56	C5 Hub	X	Free disk crosses warning/critical threshold or measurement fails.	Emit threshold warning/critical, or report UNKNOWN if measurement fails; HEALTH must not imply OK. Handler: disk monitor. Recoverable after cleanup/OS repair.

ID	Component	Category	Edge case	Design decision
EC-57	C5 Hub	I	Trusted Node sends a permanently invalid envelope.	Use signed FAILED, not a new REJECTED status, when its ID and delivery owner are valid. Origin records failure, not storage. Handler: A2 and v2 receipt verifier. Not recoverable for that publication.
EC-58	C4 Adapter	T	Many retries become due after an outage/restart.	EDF selection, per-radio serialization, an inter-frame gap and airtime gate; recheck expiry before send. Handler: radio scheduler. Recoverable only within remaining deadline/capacity.
EC-59	Configuration	I	Bind address is remote or a wildcard address.	Reject startup even when the port is valid. Handler: Builder/component loopback validator. Recoverable after editing to loopback.
EC-60	Notifications	T	Edge health heartbeat stops or connection closes.	Mark sensor state STALE and edge disconnected; keep last stopped reason. Handler: Node health cache. Recoverable after reconnect/report.
EC-61	C3 Node	I	Peer uses old delivery-policy version.	Reject compatibility before processing; upgrade peers together. Handler: v2 payload parser/trust configuration. Recoverable after coordinated upgrade.

4. Requirements traceability table

US, UC, and AC IDs come from `User_Stories.md`. NFR IDs organize the constraints and timing rules; they are not additional stakeholder approvals. P1 = must have, P2 = should have, P3 = supporting feature.

Requirement and priority	Specific design elements	Coverage, verification, or remaining gap
US-01: delivery after outage - P1	D1 A1; durable record deadline; Figures 2-3; EC-13, EC-18	Schedule no earlier than 30 s after a send; paced retries until expiry. Test reconnection with enough time for commit/receipt. The 3,600-second window includes transmission, commit, and receipt; the revised US-01 uses the same rule.
US-02: retain until storage - P1	`ACK_STORED`; Spool admission/cleanup; D1 A2; Figure 3; EC-07, EC-15, EC-53	Durable admission before success; valid storage receipt completes successfully; permanent rejection completes as FAILED. Test restart before/after admission and lost receipt. Limits: finite expiry; unadmitted Edge state is volatile.
US-03: reject invalid config - P2	Builder validator and directory publication; Figure 4; EC-30-35	Reject before deployment, leave source unchanged; test invalid fields and failed split.
US-04: stable sensor extension - P2	`SensorDriver`, registry, `DecoderRegistry`, Sensor PK; Figure 1; EC-03, EC-27	Add one driver/decoder and Sensor row; compare envelope schema before/after.
US-05: approved senders only - P2	Trust-table lookup, signature/origin checks, `Origin.expected_sender`; Figures 1-2; EC-19, EC-38	Untrusted input never reaches A2. Test unknown identity and forged origin separately.
UC-01: queued telemetry - P1	Figure 2; A1/A2; EC-08, EC-26	Main, duplicate, and unauthorized branches have decided outcomes. Run each end to end.
UC-02: configuration validation - P2	Figure 4; `validate` and `drivers`; EC-33	Main, list-driver, and unknown-driver paths have outputs and exit codes.
AC-01.1: one row, empty queue - P1	Reading PK, A2 commit, Spool cleanup; Figures 1-3; EC-15	Use one sample and one destination; reconnect before deadline with margin, then assert committed row and no pending record.
AC-01.2: duplicate gets receipt - P1	Envelope PK/content comparison; Figure 1; EC-26	Send identical envelope three times; assert one logical envelope and repeated `STORED`.
AC-01.3: unauthorized, zero rows - P1	Ingress local `UNAUTHORIZED` diagnostic; Figure 2; EC-19	Assert rejection in hub Node log and zero rows. No negative receipt; field expiry is not proof of absence.
AC-02.1: valid Linux config - P2	128-hex public identity parser; loopback/stored checks; Figure 4; EC-36, EC-45	Preserve the published identity length. Valid fixture returns status 0 with no errors.
AC-02.2: list `sim-env` once - P2	Unique driver registry and `drivers`; Figure 4; EC-34	Assert exactly one matching line in selected build.
AC-02.3: missing driver, unchanged file - P2	Semantic validator and driver-list diagnostic; Figure 4; EC-33	Hash source before/after; assert nonzero status, unknown name, and available list.

Requirement and priority	Specific design elements	Coverage, verification, or remaining gap
NFR-01: $200 budget, existing hardware - P1	D1 SQLite/local hosting; RTC provision; Figure 1; EC-54	No required paid service. The RTC and any storage purchases must fit the hardware cost ledger. We use the $200 course cap, including any RTC or storage purchase.
NFR-02: signed packets, protected local access - P1	D1 I5 signature; trust table; loopback validator; Figure 2; EC-38-39, EC-59	Test signature tampering and remote bind rejection. Protect key/config access; local processes remain trusted.
NFR-03: battery devices do not forward - P2	Builder power-role check and carrier forwarding flag; Figure 4; EC-33	Battery target requires forwarding disabled; powered relays are explicit. Verify emitted configuration and carrier startup.
NFR-04: carrier-specific packet budget - P1	A1 profile-size gate and C4 encoded-frame check; Figure 2; EC-06, EC-20	TCP wrapper <=383; LoRa wrapper <=L and final radio payload <=255. Test exact boundary and one byte above after outer encoding. No fragmentation.
NFR-05: 60 s samples, 30 s retry, 3,600 s expiry - P1	Sensor timer, A1 monotonic deadline; Figure 3; EC-02, EC-05, EC-13	Defaults are explicit; test missed ticks and exact expiry. The radio gate can delay an eligible send; expiry still takes priority.
NFR-06: Linux prototype, ESP32 target, multiple languages - P2	Protobuf contract; ESP32 local queues and LittleFS spool; Figure 3; EC-15, EC-24, EC-53	Linux is the first deployment; section 1 lists the ESP32 checks needed before field use.
NFR-07: energy and approved radio settings - P2	Forwarding disabled on battery devices; carrier airtime gate; Figures 2-3; EC-18, EC-24, EC-58	LoRa adapter must use an approved frequency/power profile and measured airtime budget. If a send is not allowed, defer without bypassing expiry. Settings and battery life require board review.
SUPPORT-01: diagnose faults - P3	Extended HEALTH, sensor/storage reports, logs; Figure 3; EC-41, EC-49-52, EC-56, EC-60	Derived from support-technician need. Read queue depth, stopped/stale sensors, clock state, and disk warnings; no dashboard or external alert service.

Reverse check. Every D0 component has a requirement: C1 -> US-04; C2 -> US-01/02; C3 -> US-01/02/05; C4 -> US-01; C5 -> US-02/04/05; C6 -> US-03. Authentication supports US-05, notifications/logging support SUPPORT-01, and configuration supports US-03/NFR-02/03. Figure 1 supports US-02/04/05, Figures 2-3 support US-01/02, and Figure 4 supports US-03.

Scope decisions. Durable storage protects US-02. Optional backup automation, multiple Hubs, and a separate UDP adapter are removed rather than assigned invented requirements. HEALTH supports fault diagnosis. D1's best-effort path remains optional and does not satisfy reliable farm delivery. A dashboard, external alert service, runtime plugin loader, extra unused decoder formats, link-mode handshake, and build-orchestration commands are outside the required release. Future adapters must earn a requirement before adding runtime behavior.

5. Design rationale

These ten decisions extend D0's DL-1 to DL-6 without repeating that log.

ID	Decision, alternative, and accepted trade-off
R-1	We chose per-Delivery durable files with workload-derived capacity over a fixed 128-record queue because three sensors need at least 183 records for the reference outage. We accepted extra flash/index space because accepted samples must survive restart. Extends DL-1/4. Changed by review PR-01/MR-01.
R-2	We chose a held sample and explicit RUNNING/BACKPRESSURED/STOPPED health states over replacing old samples or logging alone because operators need visible sampling faults. We accepted a small bounded health-report cache because silent sensor stops hide lost coverage. Changed by review PR-09.
R-3	We chose a build-time driver registry over runtime plugins because the Builder can validate the actual driver set before installation. We accepted the trade-off of rebuilding for a new driver because it keeps embedded deployment predictable. US-03/04; EC-03/34.
R-4	We chose one Hub and a peer-ID adapter over multiple-Hub and separate UDP paths because the published stories require reliable delivery to a farm Hub, not extra destinations/carriers. We accepted reduced deployment flexibility because it removes unrequired routing work. Extends DL-3. Changed by review PR-08.
R-5	We chose D1's 16-byte identity hash and 128-hex public identity over a raw 32-byte key because the published contract and AC-02.1 already agree on the former. We accepted the trade-off of trust-table lookup because verification needs the corresponding public identity. Extends DL-5; no identity-width migration is proposed.
R-6	We chose one complete output directory over independently replacing three files because a partial split could mix device configurations. We accepted the trade-off of a fresh directory and a controlled restart because deployment must select one consistent version. Clarifies I9; `hub.flags` remains a directly passed argument list.
R-7	We chose paced EDF transmission and carrier-specific packet budgets over simultaneous 30-second retry bursts and a universal 383-byte allowance because LoRa has finite airtime and a smaller single-frame payload. We accepted longer backlog delay and possible expiry under overload because radio limits cannot be bypassed. Changed by review PR-04/05/06 and MR-01/02.
R-8	We chose disk warnings and operator cleanup over unrequired automated backup rotation because the required release needs clear storage-failure behavior without a new maintenance feature. We accepted manual cleanup and no built-in disaster recovery because backups have no approved requirement. Extends DL-6. Changed by review PR-08/10.
R-9	We chose signed FAILED for permanently invalid trusted deliveries and silence for untrusted input over retrying every invalid envelope until expiry because authenticated peers need a terminal error. We accepted a coordinated policy/local-protocol v2 upgrade because old senders must not ignore rejection semantics. No REJECTED enum is introduced. Changed by review PR-03/MR-03.
R-10	We chose a battery-backed DS3231 RTC over requiring Hub connectivity or GPS for boot time because recovery must work during a network outage. We accepted an additional hardware item and bench provisioning because its battery preserves time across power loss. A missing/invalid RTC remains a visible fail-closed condition. Changed by review PR-02.

Sources checked. Pulse project resources: User Stories, D0/D1, constraints essay, and advisor record. SQLite WAL: storage concurrency. Analog Devices DS3231: I2C RTC with battery backup. Semtech SX1262: radio documentation; long-packet extensions are not part of this design.

Earlier documents. D1's prose was shortened without changing its baseline. D0 is frozen; its example UUID is not valid v4. D2 changes the queue, radio, and rejection rules and corrects I6's destination to a service Name.

6. Peer design review notes

Our review notes follow; they are also saved separately in `Peer_Review_Notes.md`.

Team: Jacob Cohen, Sujal Choukse, Yatharth Bajaj. **Recorded:** October 14, 2026.

Reviewers: Sahil, Team Street Smart; Mohammad, EpistemicRAG. We combined both reviews below. Must-fix items are due with the D2 submission on October 14; Week 8 is the course milestone specified for the optional suggestions.

Feedback from Sahil - Team Street Smart

Sahil highlighted the four-category edge-case coverage, named handlers, error paths in Figures 2-4, figure legends, and clearly stated design limits.

ID	Criterion	Issue and source	Action / resolution	Owner	Priority	Due
PR-01	Risk	128 records cannot hold three sensors' one-hour backlog. Street Smart.	Applied: sensor-derived capacity; 183 minimum and 256 reference records. Section 2.3; R-1.	Jacob Cohen	must-fix	Oct 14, 2026
PR-02	Feasibility	ESP32 recovery needs a real clock source. Street Smart.	Applied: battery-backed DS3231 RTC, bench UTC provisioning, boot checks. Section 2.3; EC-54; R-10.	Sujal Choukse	must-fix	Oct 14, 2026
PR-03	Completeness	Invalid trusted envelopes retry for an hour. Street Smart.	Applied: owned ACK and signed FAILED for permanent rejection, not a new REJECTED enum. Sections 2.3/2.5; EC-57; R-9.	Yatharth Bajaj	must-fix	Oct 14, 2026
PR-04	Risk	Backlogged retries exceed one LoRa window. Street Smart.	Applied: one radio scheduler, EDF, inter-frame spacing, airtime gate, expiry recheck. Section 2.3; Figure 3; EC-58; R-7.	Jacob Cohen	must-fix	Oct 14, 2026
PR-05	Feasibility	383 wrapper bytes do not fit a 255-byte radio payload. Street Smart.	Applied: TCP-only 383 cap; LoRa profile budget plus complete encoded payload <=255. Section 2.4; EC-20; R-7.	Jacob Cohen	must-fix	Oct 14, 2026
PR-06	Clarity	Airtime and remote-bind references are missing/wrong. Street Smart.	Applied: visible gate in Figures 2/3; NFR-07 cites EC-58; NFR-02 cites new remote-bind EC-59.	Jacob Cohen	must-fix	Oct 14, 2026

ID	Criterion	Issue and source	Action / resolution	Owner	Priority	Due
PR-07	Clarity	Draft assignment labels were unclear; figures were out of order and review notes/decision markers were missing. Street Smart.	Applied: label D2 as Assignment 7 within the Assignment 4 collection; order Figures 1-4; add part 6 and rationale markers.	Yatharth Bajaj	must-fix	Oct 14, 2026
PR-08	Completeness	Backups, multiple Hubs, UDP have no requirement. Street Smart.	Applied early (Oct 14): remove all three optional runtime features; one Hub/TCP baseline. Sections 2.2/2.4/2.5 and part 4; R-4/8.	All team members	worth con sidering	Week 8
PR-09	Clarity	Stopped sensors are absent from HEALTH. Street Smart.	Applied early (Oct 14): typed sensor reports, stopped reason, disconnected/stale states. Sections 2.2/2.3; EC-60; R-2.	Yatharth Bajaj	worth con sidering	Week 8
PR-10	Risk	No low-disk warning; sensor-count assumption unclear. Street Smart.	Applied early (Oct 14): warning/critical thresholds and HEALTH; size both D1's 10 sensors and new 30-sensor case. Section 2.5; EC-56; R-8.	Yatharth Bajaj	worth con sidering	Week 8

Feedback from Mohammad - EpistemicRAG

Mohammad praised the repository organization, project vision, core modules, and functional breakdown. He asked for more quantitative hardware and latency limits, explicit disruption recovery, and concrete API schemas. We assigned an owner and a submission deadline to each action.

ID	Criterion	Issue and source	Action / resolution	Owner	Priority	Due
MR-01	Feasibility	Hardware/resources and heavy-load latency need explicit budgets. EpistemicRAG.	Applied: resource/latency table, rate/capacity calculations, and a radio overload rejection rule. Section 2.8; R-1/7.	Jacob Cohen	must-fix	Oct 14, 2026
MR-02	Risk	Network and dependency delays need concrete recovery rules. EpistemicRAG.	Applied: backoff and held samples, paced retry, storage rollback, terminal rejection, clock fault handling. Sections 2.2-2.5; EC-08/28/54/57/58.	Yatharth Bajaj	must-fix	Oct 14, 2026
MR-03	Clarity	Module integration needs explicit request/error schemas. EpistemicRAG.	Applied: version-2 health/report/ACK contracts, policy version, types, ownership, errors, and compatibility rules. Sections 2.2-2.5; EC-61; R-9.	Jacob Cohen	must-fix	Oct 14, 2026

Review summary

Most feedback: Risk and clarity tied, with four action items each. Feasibility had three and completeness had two. The main concerns were whether backlog and radio limits could satisfy the delivery promise, and whether the diagrams and contracts described the same behavior. All seven Street Smart must-fix items and the three EpistemicRAG action items have corresponding design changes above; we applied the three Week 8 suggestions early, on October 14.

Top two issues we raised while reviewing Street Smart

1. **Risk - unblurred images in the unresolved-location cache.** Street Smart states that photos are blurred before storage, but its unresolved-location path saves a detection locally before blurring. Faces and license plates could reach the worker's disk. We suggested storing metadata only and extracting the image from the video later, or blurring the image before saving it.

2. **Completeness - contradictory detection counts.** One section keeps the top 10 detections per frame, while the pipeline selects only the highest-confidence box and the database permits one detection per frame. A frame with two potholes could lose one without warning. We asked the team to choose one design: explicitly keep one detection and state the limitation, or support several and update the database key and related sections.

Review-driven decisions: R-1, R-2, R-4, R-7, R-8, R-9, and R-10 are marked in part 5. D0 is unchanged. D1 has editorial corrections; D2 records the revised capacity, carrier limits, and rejection policy.